

INTRODUCTION HTML5 ET CSS3

Table des matières

Introduction au langage HTML5 6

À propos du Consortium World Wide Web (W3C)	6
À propos du Web Hypertext Application Technology Working Group (ou WHATWG).	6
Voici un exemple du code d'une page Web en HTML.	7
Les versions de CSS	10
Définition des différentes balises	11
Etude plus en détail de certaines balises communément utilisées	26
Structure de base d'une page en HTML5 avec les éléments de section	26
Présentation d'une structure de page plus élaborée	27
Détails des éléments de section HTML5	28
Pour rendre l'HTML5 compatible avec IE avant la version 9.	29
Exercice	29
L'élément HTML <audio>	30
L'élément HTML <video>	32
Les différents éléments du formulaire	33
Balise <datalist>	35
Attributs importants pour les éléments du formulaire	37
1. Placeholder	37
2. Required	38
3. Pattern	38
4. Contenteditable	38
5. Form	39
6. Novalidate	39
7. draggable, dropzone	39
8. async	39
9. reversed	39
10. download	40
11. autocomplete	40

12. dirname	40
13. min, max, step	40
14. Attributs personnalisés	41
Balise <details>	41
Balise <embed>	41
Balise <figure>	42
Balise <figcaption>	42
Balise <mark>	42
Balise <canvas>	43
Balise <SVG>	45
Le Scalable Vector Graphics (SVG)	45
Inkscape	45

Introduction au langage CCS3	46
Les nouveaux sélecteurs CSS3	46
1.Les sélecteurs d'attributs	46
[attr^="stringValue"]	46
[attr\$="stringValue"]	47
[attr*="stringValue"]	48
2.Le combinateur d'adjacence directe	49
3.Les pseudo-classes	50
:nth-last-child(expression)	53
:last-child	55
:nth-last-of-type(expression)	55
:first-of-type	55
:last-of-type	55
:only-child	55
:checked	55
:empty	55
:target	56
:valid	56
:invalid	56
Les pseudo-éléments	57
::first-line	57
::first-letter	57
::selection	58
::before et ::after	58
Propriété de style pour le texte	60
La taille	60
La police	61
Le Gras	61
Couper un mot avec la propriété word-wrap	61
Dépassement du texte avec la propriété text-overflow	62
Les ombres sur le texte	63
Soulignement et "décorations"	63
L'alignement du texte	63
La transparence	66
La notation HSLa	67
Créer des ombres sur des boîtes	69
Arrière-plans multiples	70
Modifier la taille d'un arrière plan	70
Les filtres CSS3	73
La fonction calc() en CSS3	75
Compatibilité	75
A propos des unités de mesure	76
Réaliser une transition	82
La propriété transition-property	82
La propriété transition-duration	82
La propriété transition-timing-function	82
La propriété transition-delay	83
La notation raccourcie transition	84
Les transformations	86
1- La rotation	86
2- La translation	86
3- Le redimensionnement	87
La distorsion	88
Les animations	89

Les propriétés	89	
Fonctionnement des fonctions de timing -•transition-timing-function		91
La fonction CSS cubic-bezier()	92	
Outil permettant de réaliser des courbes de Bézier en ligne		93
La fonction steps()	94	
A propos de CSS3 Grid Layout	95	
Réaliser un fond dégradé sans image	96	
Mise en place des media queries	97	
Appliquer une media query	98	
Chargement d'une feuille de style différente	98	
Le meta viewport pour les mobiles	100	
Flexbox	101	
display : box	101	
Empilement vertical ou horizontal	103	
Ordre d'empilement	104	
Flexibilité : remplir l'espace	105	
Tableau des compatibilités	107	
Mémento des propriété CSS3	108	
Propriétés du texte	108	
Propriétés de couleur et de fond	110	
Propriétés des boîtes	111	
Propriétés de positionnement et d'affichage	114	
Propriétés des listes	115	
Propriétés des tableaux	116	
Autres propriétés	117	

Introduction au langage HTML5

L'**HTML5** est une *évolution* de l'HTML 4.01 (et de l'XHTML 1.0).

Cette évolution consiste en une multitude de nouvelles fonctionnalités qui ont été apportées au langage HTML.

HTML5 nous propose une certaine simplification du code et tente d'atteindre l'interopérabilité.

Le langage **HTML5** est le résultat du travail conjoint du W3C et du WHATWG

À propos du Consortium World Wide Web (W3C)

Le Consortium World Wide Web (W3C) est un consortium international dont les organisations membres, une équipe à plein temps et le public collaborent au développement de standards Web. Le W3C poursuit sa mission essentiellement par la création de standards Web et de directives visant à assurer au Web une croissance à long terme. Plus de 325 organisations adhèrent au consortium. Le W3C est piloté conjointement par le Laboratoire d'Informatique et d'Intelligence Artificielle du MIT (MIT CSAIL) aux États-Unis, le Groupement Européen de Recherche en Informatique et en Mathématiques (ERCIM) basé en France et l'Université de Keio au Japon, et possède plusieurs bureaux dans le monde. Pour plus d'informations, consultez l'adresse suivante : <http://www.w3.org/>

À propos du Web Hypertext Application Technology Working Group (ou WHATWG).

Le WHATWG est une collaboration non officielle des différents développeurs de navigateurs web ayant pour but le développement de nouvelles technologies destinées à faciliter l'écriture et le déploiement d'applications à travers le Web. La liste de diffusion du groupe de travail est publique et ouverte à tous. La Mozilla Foundation, Opera Software et Apple, Inc. en sont les premiers contributeurs.

Voici un exemple du code d'une page Web en HTML.

Avant

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr" >
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<link rel="stylesheet" type="text/css" href="design.css" />
</head>
```

Maintenant

```

<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="utf-8" />
<link rel="stylesheet" href="design.css" />
</head>
<body>

    <header class="ma-class-en-css">
        <!-- menu du haut -->
        <nav class="ma-class-en-css">
            <ul>
                <li><a href="mon-url">nom de mon lien</a></li>
                <li><a href="mon-url">nom de mon lien</a></li>
                <li><a href="mon-url">nom de mon lien</a></li>
            </ul>
        </nav>
    </header>

    <!-- contenu de ma page -->
    <div>
        <!-- article n°1 -->
        <article class="ma-class-en-css">
            <h1>titre de mon article</h1>
            <p>texte de mon article</p>
            <section class="ma-class-en-css">
                <h2>sous-titre mon article</h2>
                <p>texte de mon sous-titre</p>
                <figure class="ma-class-en-css">
                    <a href="mon-url"></a>
                </figure>
            </section>
        </article>
    </div>

    <footer class="ma-class-en-css">
        <!-- menu du bas -->
        <nav class="ma-class-en-css">
            <ul>
                <li><a href="mon-url">nom de mon lien</a></li>
                <li><a href="mon-url">nom de mon lien</a></li>
                <li><a href="mon-url">nom de mon lien</a></li>
            </ul>
        </nav>
    </footer>
</body>
</html>

```

Les différentes versions de HTML et CSS

Au fil du temps, les langages HTML et CSS ont beaucoup évolué.
Dans la première version de HTML il n'était pas possible d'afficher des images!

Voici un bref historique de ces langages.

Les versions de HTML

HTML 1 : c'est la toute première version créée par Tim Berners-Lee en 1991.

HTML 2 : la deuxième version du HTML apparaît en 1994 et prend fin en 1996 avec l'apparition du HTML 3.0. C'est cette

version qui posera en fait les bases des versions suivantes du HTML. Les règles et le fonctionnement de cette version

sont donnés par le W3C (tandis que la première version a été créée par un seul homme).

HTML 3 : apparue en 1996, cette nouvelle version du HTML rajoute de nombreuses possibilités au langage comme les

tableaux, les applets, les scripts, le positionnement du texte autour des images, etc.

HTML 4 : il s'agit de la version la plus répandue du HTML (plus précisément, il s'agit de HTML 4.01). Elle apparaît pour la

première fois en 1998 et propose l'utilisation de frames (qui découpent une page web en plusieurs parties), des tableaux

plus complexes, des améliorations sur les formulaires, etc. Mais surtout, cette version permet pour la première fois

d'exploiter des feuilles de style, notre fameux CSS !

HTML 5 : c'est la dernière version. Elle fait beaucoup parler d'elle car elle apporte de nombreuses améliorations comme la possibilité d'inclure facilement des vidéos, un meilleur agencement du contenu, de nouvelles fonctionnalités pour les formulaires, etc.

C'est cette version que nous allons découvrir ensemble.

Les versions de CSS

CSS 1 : dès 1996, on dispose de la première version du CSS. Elle pose les bases de ce langage qui permet de présenter sa

page web, comme les couleurs, les marges, les polices de caractères, etc.

CSS 2 : apparue en 1999 puis complétée par CSS 2.1, cette nouvelle version de CSS rajoute de nombreuses options. On peut désormais utiliser des techniques de positionnement très précises, qui nous permettent d'afficher des éléments où on le souhaite sur la page.

CSS 3 : c'est la dernière version, qui apporte des fonctionnalités particulièrement attendues comme les bordures arrondies, les dégradés, les ombres, les transitions, les animations, etc.

A propos de la permissivité de HTML5

HTML 5, en tant que successeur de HTML 4.01, conserve par défaut la permissivité syntaxique de son prédécesseur : majuscules autorisées pour les noms d'éléments et d'attributs, apostrophe simple ou double non obligatoire pour signaler les valeurs.

En outre, HTML 5 poursuit les libertés de son ancêtre. Ainsi, les éléments `<p>`, `<dd>`, `<dt>`, `<li>`, `<optgroup>`, `<option>`, `<td>`, `<th>`, `<tr>`, `<thead>` et `<tfoot>` ne nécessitent pas de balise fermante pour être valides.

Seule la version XHTML 5 oblige à fermer ces éléments.

Mieux encore, certains éléments ne nécessitent ni balise fermante ni balise ouvrante. C'est le cas de `<html>`, `<head>`, `<body>`, `<thead>`, `<tfoot>` et `<tbody>`. Cela signifierait que la présence même de ces éléments devient implicite.

Pour finir, notons que l'attribut `type` (rencontré principalement au sein des balises `<script>` et `<style>`) devient superflu.

Le code HTML suivant – parfaitement valide et suffisant – résume à lui seul la concision autorisée en HTML 5 :

```
<!doctype html>
<meta charset=UTF-8>
<title>titre de page</title>
<p>hello world !
```

Cependant, je vous invite à conserver autant que possible la rigueur de XHTML au sein de vos documents.

Définition des balises

(HTML5 W3C Recommendation 28 October 2014)

Balise	Description
<code><!--...--></code>	Commentaire dans le document HTML
<code><!DOCTYPE></code>	L'inclusion du DOCTYPE dans un document HTML assure que le navigateur interprétera la version HTML ainsi déclarée. En HTML5 le doctype à déclarer est : <code><!DOCTYPE html></code> .
<code><a></code>	Utilisée pour les hyperliens. Attention : l'attribut "name" n'existe plus pour les liens. Attribut(s) spécifique(s)
<code><abbr></code>	Pour une abréviation. Son attribut "title" ne doit être utilisé que pour l'extension de l'abréviation et rien d'autre. Une nouvelle version d' <code><abbr title="Hypertext Markup Language">HTML</abbr></code> est enfin sortie.
<code><acronym></code>	Non utilisé
<code><address></code>	Généralement utilisée dans le conteneur <code><footer></code> , cette balise est utilisée à la place de la balise <code><p></code> , les informations nécessaires pour contacter la ou les auteurs du document cité. Cela peut-être son nom ou un label suivi de son adresse email par-exemple ou adresse postale. Confusion possible : on utilisera la balise <code><p></code> pour donner l'adresse postale d'une entreprise sans relation avec l'auteur du document s'y rapportant. Exemple: <pre> <address> Ecrit par Marc Albert : Contactez-moi </address> </pre>

Balise	Description
<applet>	Non utilisé
<area>	Définit une zone ou plusieurs zones pour une carte image. Cette balise s'utilise toujours avec la balise <map>
<article>	Utilisé pour du contenu ayant son propre sens indépendamment du reste des autres éléments de la page, ce contenu est distribuable et réutilisable. Cela peut-être un billet de forum, un article de journal, un gadget, un commentaire d'utilisateur... La balise <article> peut avoir son propre header et footer. Confusion possible avec la balise <section> qui regroupe des éléments de thématique identique. Exemple: <pre> <article> La grande messe sur l'utilisation des standards du Web a démontré que (...) </article> ou <article> <p> La grande messe sur l'utilisation des standards du Web a démontré que (...)</p> </article> </pre>
<aside>	Balise de structure supposant avoir un titre de type <h1>. Cette balise permet de regrouper des informations non essentielles relatives au site Web. Lorsque la balise <aside> se trouve dans un article, son contenu se réfère à l'article lui même et non au site Web (par exemple des notes de pages, un glossaire ou tout élément relatifs à l'article).

<audio>	L'élément HTML <code><audio></code> est utilisé pour rajouter un contenu sonore dans un document. Il peut contenir plusieurs sources audio introduites avec l'attribut <code>src</code> ou l'élément <code><source></code>. Le navigateur décidera lequel utiliser. Du contenu alternatif peut aussi être ajouté pour les navigateurs ne supportant pas cette fonctionnalité. Un contenu alternatif peut être ajouté pour les navigateurs ne supportant pas l'élément <code><audio></code>. Attributs: autoplay Un attribut booléen. Si celui-ci est renseigné (même avec une valeur "false"), l'élément audio sera joué automatiquement dès qu'assez de données auront été chargées pour que la lecture ne soit pas interrompue. controls Si l'attribut est présent, le navigateur offrira à l'utilisateur des contrôles pour qu'il puisse gérer la lecture, le volume, la navigation, et la pause ou la reprise de la lecture. loop Un attribut booléen. S'il est renseigné, la lecture du fichier se fera en boucle.. muted Un attribut booléen, indiquant si le son de l'élément audio doit être coupé. Sa valeur par défaut est false, indiquant que l'élément audio doit être joué. preload Indique si la musique peut être préchargée dès le chargement de la page ou non. Cet attribut peut prendre les valeurs : auto (par défaut) : le navigateur décide s'il doit précharger toute la musique, uniquement les métadonnées ou rien du tout. metadata : charge uniquement les métadonnées (durée, etc.). none : pas de préchargement. Utile si vous ne voulez pas gaspiller de bande passante sur votre site.. src L'URL du fichier audio à inclure.
----------------------	---

Balise	Description
	Utilisé dans un paragraphe pour une partie de texte en gras, sémantiquement faible, on lui préférera la balise pour donner de l'importance au texte.
<base>	Cette balise permet de définir l'URL de base (l'utilisation de cette balise est controversée).
<basefont>	Non utilisé
<bdo>	La balise <bdo> modifie le sens de l'écriture. L'hébreu s'écrit par exemple de la droite vers la gauche. L'attribut spécifique est dir dont les deux valeurs sont: - ltr : pour un texte allant de gauche à droite (<i>left-to-right</i>). - rtl : pour un texte allant de droite à gauche (<i>right-to-left</i>). - auto : Selon la nature du contenu, le navigateur décide de la direction. Exemple: <pre><bdo dir="rtl"> Ce texte sera affiché comme s'il était écrit de la droite vers la gauche </bdo></pre>
<bdi>	L'élément HTML <bdi> (ou élément d'isolation de texte bi-directionnel) isole une portée (<i>span</i>) de texte pouvant être formaté dans une direction différente de celle du texte qui l'entoure. Remarque: pas supporté par IE, OPERA et Safari.
<big>	Non utilisé

Balise	Description
<blockquote> >	Pour une longue citation venant d'une source externe, l'adresse de la source peut-être indiquée dans l'attribut "cite". Dans le cas d'un forum de discussion ou d'un blog, un utilisateur peut utiliser cette balise pour citer le commentaire d'un autre utilisateur. Nous n'utiliserons pas cette balise pour un dialogue. Confusion possible avec la balise <u><q></u>. Exemple: <pre><blockquote cite="http://www.monsite.be/article5"> <p> L'élément BLOCKQUOTE indique une citation longue (avec un contenu de type bloc) et l'élément Q est censé être utilisé pour des citations courtes (avec un contenu de type en-ligne) ne nécessitant pas un saut de paragraphe. </p> <p>L'utilisation de l'élément BLOCKQUOTE pour obtenir le retrait d'un texte est déconseillée en faveur de l'utilisation de feuilles de style. </p> </blockquote></pre>
<body>	Partie principale du document HTML

	Utilisée dans un paragraphe, cette balise permet de créer un saut de ligne.
<button>	Pour un bouton. il peut être utilisé comme commande.
<canvas>	Pour créer un graphique avec la possibilité de l'animer avec du JavaScript.
<caption>	Pour le titre d'un tableau. Note : En HTML5, l'attribut "summary" de la balise "table" n'est plus valide et ne doit pas être utilisé.
<center>	Non utilisé

Balise	Description
<cite>	Utilisée pour le titre d'une oeuvre, d'un document ou d'un évènement. Cette balise peut-être utilisée conjointement avec la balise <u><q></u> . Cette balise ne peut être utilisée pour une personne.
<code>	Pour déclarer du code informatique. On peut l'utiliser à l'intérieur de la balise <code><pre></code> pour du texte pré formaté. On recommande d'utiliser une classe avec le nom du langage informatique cité dans cette balise. On peut ajouter un classe pour décrire le type de langage utilisé dans la balise exemple : <code><code class="language-javascript"></code> . Confusion possible avec la balise <code><samp></code> .
<col>	Utilisé pour créer des colonnes dans un tableau
<colgroup>	Utilisé pour créer des groupes de colonnes dans un tableau.
<command>	Cette balise est utilisée comme un bouton, un radiobutton, ou un checkbox. Elle ne peut être visible qu'à l'intérieur de la balise <code><menu></code> , dans le cas contraire, elle ne sera pas visible et utilisée comme raccourcis clavier. Attention, l'attribut "title" à une valeur sémantique différente pour cette balise.
<datalist>	Pour une liste déroulante.
<dd>	Pour la description d'une définition, s'utilise avec la balise <code><dl></code> .
	Utilisé pour indiquer qu'une partie de texte est supprimée, mais conservée pour en garder la trace, on la stylisera en CSS. On peut utiliser cette balise conjointement avec la balise <code><ins></code> pour indiquer le nouveau texte qui est à prendre en compte. Confusion possible avec la balise <code><s></code> qui indique qu'un texte n'est pas correct ou non pertinent.
<details>	Pour apporter des détails sur Widget, il peut être utilisé pour cacher/afficher des informations complémentaires. Il peut être le conteneur de la balise <code><summary></code> .

Balise	Description
<dfn>	Représente le terme d'une définition, on l'utilise dans un paragraphe ou une liste de définitions. On peut utiliser cette balise conjointement avec <abbr>. Dans la même section, il est possible de créer un lien pointant sur cette définition, utile pour la création d'infobulle par exemple.
<dir>	Non utilisé
<div>	Balise sans valeur sémantique réelle, elle sert de conteneur pour une mise en forme en CSS. Cette balise doit être utilisée en dernier recours, lorsqu'aucune autre balise ne peut convenir.
<dl>	Pour une liste de définitions. Confusion possible : cette balise ne peut être utilisée pour créer des dialogues, les balises <p> restant appropriées.
<dt>	Pour déclarer une définition, s'utilise avec la balise <dl>.
	Utilisé dans un paragraphe pour mettre une partie de texte en emphase.
<embed>	Utilisé pour du contenu externe et interactif ou pour un plug-in.
<fieldset>	Pour regrouper des éléments d'un formulaire.
<figcaption>	Utilisée dans le conteneur <figure>, avant ou après le contenu, cette balise permet d'écrire une légende ou une description.

Balise	Description
<figure>	Utilisée pour regrouper un ou plusieurs médias (illustrations, diagrammes, exemples de code, photos...) attachés au document sans pour autant en suivre le flux . Les images s'y trouvant n'ont pas obligation d'être inséré dans une balise <p> et dans le cas d'une liste d'images, on peut omettre les listes à puce. On utilisera la balise figcaption pour décrire les médias utilisés. Confusion : cette balise ne doit être utilisée dès qu'il y a une image dans un article, on peut aussi la confondre avec la balise <aside>.
	Non utilisé
<footer>	Regroupe des informations de bas de page dans une section ou un article. Cette balise permet d'ajouter des liens de navigation sans utilisation de la balise <nav>.
<form>	Pour un formulaire.
<frame>	Non utilisé
<frameset>	Non utilisé
<h1> à <h6>	Utilisés pour la hiérarchisation des titres.
<head>	Pour les informations d'en tête du document HTML.
<header>	Pour l'entête d'une section et/ou d'une page, cette balise est utile pour une introduction et/ou des éléments de navigation. Cette balise peut-être utilisée dans la balise <section> et dans la balise <article>.
<hgroup>	Regroupe des titres de niveau Hn dans la balise <head> (à noter : cette balise a été supprimée puis réintégrée, son avenir est incertain.).
<hr>	Permet de créer une ligne de séparation.
<html>	Déclaration du document HTML. On lui rajoutera l'attribut lang pour déclarer la langue utilisée.

Balise	Description
<i>	Utilisée dans un paragraphe pour indiquer qu'une partie du texte est différent du reste. Il ne faut pas utiliser cette balise lorsque , <cite>, <dfn>, , <q>, <small> ou peuvent être utilisées. Note : Certains analyseurs de site nous disent que cette balise est obsolète, ce qui est totalement faux.
<iframe>	Pour créer une sous-fenêtre.
	Pour déclarer une image.
<input>	Pour un champ de texte.
<ins>	Pour insérer un nouveau texte dans un document révisé. On peut utiliser cette balise avec qui permet d'indiquer qu'une portion de texte n'est plus valable.
<keygen>	L'élément keygen représente un contrôle de générateur de clé stockée dans "keystore local".
<kbd>	Lorsque le texte est utilisé pour définir des actions au clavier.
<label>	Étiquette utilisée comme titre d'une commande.
<legend>	Titre du fieldset auquel il se rapporte.
	Item d'une liste ordonnée ou à puce.
<link>	Permet de lier une ressource externe à la page HTML. Attention l'attribut "title" à une valeur sémantique différente.
<map>	Utilisée pour créer une carte d'image avec des zones réactives. L'attribut "name" est obligatoirement requis.
<mark>	Pour marquer du texte, par exemple surligner un résultat de recherche. On stylisera cette balise en CSS.

Balise	Description
<menu>	Pour une liste de commande. Confusion possible avec la balise <nav>.
<meta>	Permet d'ajouter des métas à la page HTML.
<meter>	Pour les mesures.
<nav>	Pour regrouper des liens qu'ils soient internes à la page ou pour des pages liées. Il est recommandé, mais non obligatoire d'utiliser les listes à puce pour lister les liens.
<noframes>	Non utilisé
<noscript>	Utilisée pour indiquer un message dans le cas où JavaScript serait désactivé.
<object>	Pour déclarer un objet.
	Utilisé pour les listes ordonnées.
<optgroup>	Pour grouper des informations dans une liste déroulante.
<option>	Pour déclarer un item dans une liste déroulante. Peut-être utilisé comme <u>"commands"</u> .
<output>	Représente le résultat d'un calcul.
<p>	Définit un paragraphe contenant une ou plusieurs phrases. Cette balise ne doit pas être utilisée si une autre balise est mieux indiquée.
<param>	Pour paramétrer un objet.
<pre>	Ecrire un texte préformaté. L'utilisation de la balise <p> n'est pas obligatoire. Peut-être lui même le conteneur de la balise <code>.

Balise	Description
<progress>	Pour une barre de progression.
<q>	Utilisée pour une citation courte provenant d'une ressource externe. Si l'on connaît l'URL de la source de la citation, on pourra l'indiquer grâce à l'attribut "cite". Dans tous les autres cas, on utilise les guillemets sans balise spécifique. On peut aussi utiliser la balise <cite> pour citer un auteur. Confusion possible avec la balise <u><blockquote></u> .
<rp>	Utilisé en annotations ruby pour définir ce qui est à montrer aux navigateurs ne supportant les éléments ruby.
<rt>	Pour expliquer des annotations en Ruby.
<ruby>	Utilisé pour des annotations en Ruby.
<s>	Utilisé pour identifier une partie de texte qui n'est pas correct ou non pertinent. Confusion possible avec la balise qui permet d'indiquer qu'un texte doit être remplacé ou supprimé.
<samp>	Utilisée dans un paragraphe pour écrire un échantillon de code. Confusion possible avec la balise <code>.
<script>	Pour ajouter un script internet ou externe.
<section>	Utilisée pour regrouper des éléments différents, mais partageant la même thématique Cette balise est le plus souvent utilisé avec un header.
<select>	Pour une liste déroulante.
<small>	(Attention la balise a été redéfinie, elle ne sert plus à minimiser un texte). La balise small est utilisée comme contenu relatif mais non essentiel. On utilisera pour déclarer un copyright, des disclaimers, des mises en garde...

Balise	Description
<source>	Utilisée dans les balises <video> et <audio> pour indiquer l'url et le type des médias.
	Utilisée pour mettre en style une portion de texte qui se différencie des autres. Cette balise ne doit pas être utilisée si une autre balise de formatage de texte convient mieux.
<strike>	Non utilisé, la balise del remplace cette balise.
	Utilisé dans un paragraphe pour mettre un texte en gras.
<style>	Permet de définir un style dans le document HTML. Attention, l'attribut title à une valeur sémantique différente pour cette balise.
<sub>	Déclarer un indice : Petit caractère placé en bas et à droite d'un autre caractère.
<summary>	Généralement utilisé à l'intérieur du conteneur <detail>, il permet de décrire le sommaire, la légende ou le titre d'un élément.
<sup>	Déclarer un exposant : petit caractère placé en haut et à droite d'un autre caractère.
<table>	Pour un tableau. Note : En HTML5, l'attribut summary utilisé en accessibilité n'est plus valide et ne doit pas être utilisé : la balise <caption> devient indispensable.
<tbody>	Utilisé pour le corps d'un tableau.
<td>	Pour déclarer une cellule dans un tableau.
<textarea>	Pour a champ de saisi.
<tfoot>	Utilisé pour le pied de page d'un tableau.

Balise	Description
<code><th></code>	Pour déclarer une cellule dans l'entête d'un tableau.
<code><thead></code>	Utilisé pour l'entête d'un tableau.
<code><time></code>	Pour déclarer une date ou une heure.
<code><title></code>	Pour le titre de la page HTML en cours de lecture.
<code><tr></code>	Pour déclarer un champ dans un tableau.
<code><track></code>	Barre pour indiquer une portion de temps.
<code><tt></code>	Non utilisé
<code><u></code>	Non utilisé, cette balise crée une confusion avec la balise <code><a></code> au niveau des styles
<code></code>	Pour créer une liste à puces.
<code><var></code>	Pour déclarer une variable.
<code><video></code>	Pour lire une video.
<code><wbr></code>	Balise non fermante à utiliser à l'intérieur d'un mot long pour forcer un retour à la ligne.
<code><xmp></code>	Non utilisé

Etude plus en détail de certaines balises communément utilisées

Structure de base d'une page en HTML5 avec les éléments de section

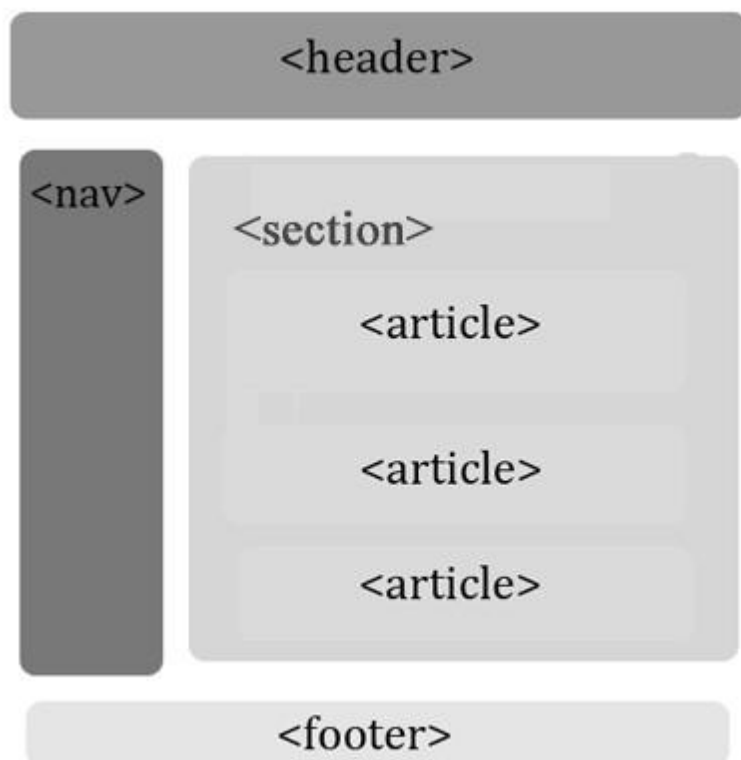
```

<!DOCTYPE HTML>
<html>

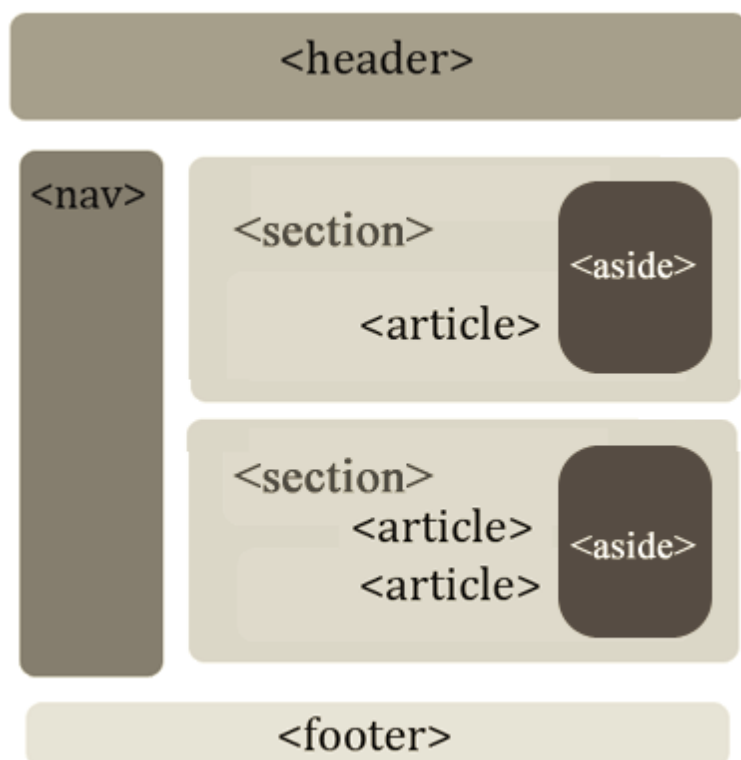
<head>
  <title>Titre de la page</title>
  <meta charset="utf-8">
</head>

<body>
  <div id="page">
    <!-- DIV : Aucun sens sémantique – container général -->
    <header> Entête de la page </header>
    <nav>Menu de la page </nav>
    <aside>Liens lié à l'élément qui le borde </aside>
    <section>
      <article> Contenu - Article 1 </article>
      <article> Contenu - Article 2</article>
      <article> Contenu - Article 3 </article>
    </section>
    <footer> Pied-de-page de la page </footer>
  </div>
</body>
</html>

```



Présentation d'une structure de page plus élaborée



Exemple d'une structure d'un article



Détails des éléments de section HTML5

Le tableau suivant récapitule les nouveaux éléments de section et leur usage le plus commun, tel que décrit par la spécification.

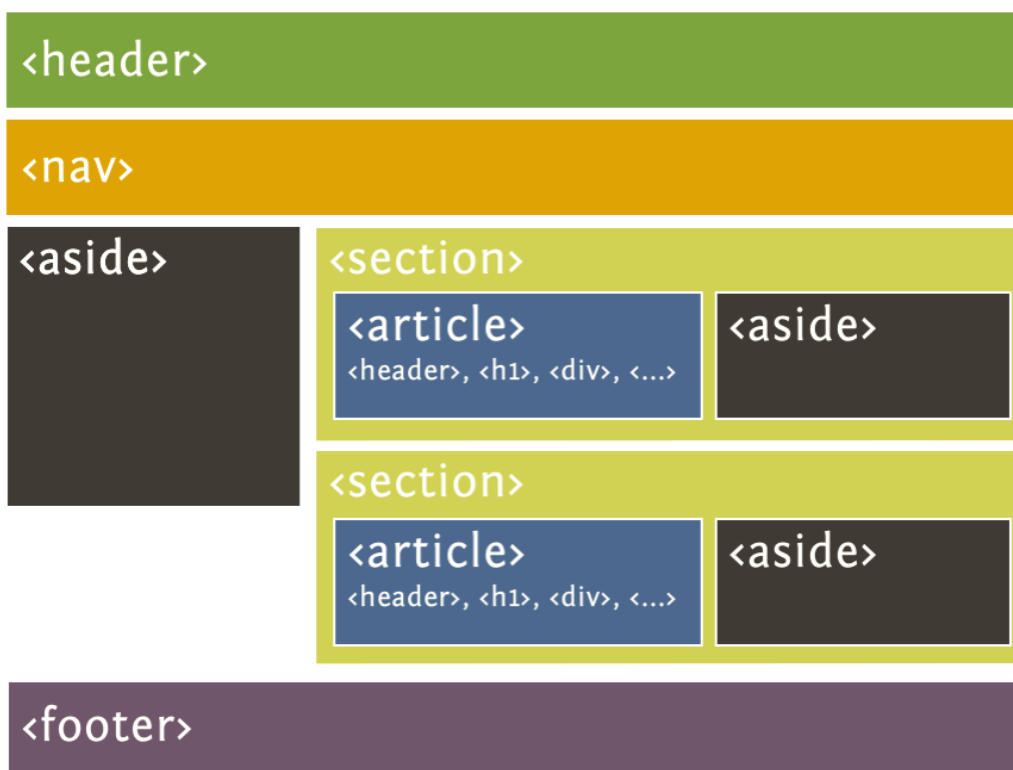
Nom	Détails
<section>	Section générique regroupant un même sujet, une même fonctionnalité, de préférence avec un en-tête
<article>	Section de contenu indépendante, pouvant être extraite individuellement du document ou syndiquée (flux RSS ou équivalent), sans pénaliser sa compréhension
<nav>	Section possédant des liens de navigation principaux (au sein du document ou vers d'autres pages)
<aside>	Section dont le contenu est un complément par rapport à ce qui l'entoure, qui n'est pas forcément en lien direct avec le contenu mais qui peut apporter des informations supplémentaires.
<header>	Section d'introduction d'un article, d'une autre section ou du document entier (en-tête de page).
<footer>	Section de conclusion d'une section ou d'un article, voire du document entier (pied de page).
<main>	Section principale, reprenant le contenu de la page. Un document ne peut pas avoir plus d'un seul élément <code><main></code>

Pour rendre l'HTML5 compatible avec IE avant la version 9.

Vous devez ajouter dans le head, la ligne de code suivante:

```
<!--[if lt IE 9]>
<script src="//html5shiv.googlecode.com/svn/trunk/html5.js">
</script>
<![endif]-->
```

Exercice: réalisez la figure suivante en HTML5 et CSS



L'élément HTML <audio>

L'élément HTML <audio> est utilisé pour rajouter un contenu sonore dans un document. Il peut contenir plusieurs sources audio introduites avec l'attribut src ou l'élément <source>. Le navigateur décidera lequel utiliser. Du contenu alternatif peut aussi être ajouté pour les navigateurs ne supportant pas cette fonctionnalité.

Un contenu alternatif peut être ajouté pour les navigateurs ne supportant pas l'élément <audio>.

Attributs:

Comme tous les autres éléments HTML, cet élément possède les attributs globaux.

autoplay

Un attribut booléen. Si celui-ci est renseigné (même avec une valeur "false"), l'élément audio sera joué automatiquement dès qu'assez de données auront été chargées pour que la lecture ne soit pas interrompue.

controls

Si l'attribut est présent, le navigateur offrira à l'utilisateur des contrôles pour qu'il puisse gérer la lecture, le volume, la navigation, et la pause ou la reprise de la lecture.

loop

Un attribut booléen. S'il est renseigné, la lecture du fichier se fera en boucle..

muted

Un attribut booléen, indiquant si le son de l'élément audio doit être coupé. Sa valeur par défaut est false, indiquant que l'élément audio doit être joué.

preload

Indique si la musique peut être préchargée dès le chargement de la page ou non.

Cet attribut peut prendre les valeurs :

auto (par défaut) : le navigateur décide s'il doit précharger toute la musique, uniquement les métadonnées ou rien du tout.

metadata : charge uniquement les métadonnées (durée, etc.).

none : pas de préchargement. Utile si vous ne voulez pas gaspiller de bande passante sur votre site..

src

L'URL du fichier audio à inclure. Cet attribut est sujet aux spécifications du CORS. Il est optionnel et il est possible d'utiliser l'élément <source> dans le bloc de cet élément pour définir les fichiers audio à intégrer.

Exemples

<!-- Simple lecture audio -->

<audio src="Audio.ogg" autoplay>

Votre navigateur ne supporte pas l'élément `<code>audio</code>`.

</audio>

<!-- Lecture Audio avec sous-titres -->

<audio src="foo.ogg">

<track kind="captions" src="foo.en.vtt" srclang="en" label="English">

<track kind="captions" src="foo.sv.vtt" srclang="sv" label="Svenska">

</audio>

Élément audio utilisant l'élément `<source>` :

<audio controls="controls">

Votre navigateur ne supporte pas l'élément `<code>audio</code>`.

<source src="foo.wav" type="audio/wav">

</audio>

L'élément HTML <video>

La balise **<video>** est reconnue par tous les navigateurs récents, y compris Internet Explorer à partir de la version 9 (IE9).

Il suffit d'une simple balise **<video>** pour insérer une vidéo dans la page :

```
<video src="mavideo.webm"></video>
```

Rajoutons quelques attributs (la plupart sont les mêmes que pour la balise **<audio>**) :

poster : image à afficher à la place de la vidéo tant que celle-ci n'est pas lancée. Par défaut, le navigateur prend la première image de la vidéo mais, comme il s'agit souvent d'une image noire ou d'une image peu représentative de la vidéo, je vous conseille d'en créer une ! Vous pouvez tout simplement faire une capture d'écran d'un moment de la vidéo.

controls : pour ajouter les boutons « Lecture », « Pause » et la barre de défilement. Cela peut sembler indispensable, mais certains sites web préfèrent créer eux-mêmes leurs propres boutons et commander la lecture avec du JavaScript. En ce qui nous concerne, ce sera largement suffisant !

width : pour modifier la largeur de la vidéo.

height : pour modifier la hauteur de la vidéo.

loop : la vidéo sera jouée en boucle.

autoplay : la vidéo sera jouée dès le chargement de la page. Là encore, évitez d'en abuser, c'est en général irritant d'arriver sur un site qui lance quelque chose tout seul !

preload : indique si la vidéo peut être préchargée dès le chargement de la page ou non.

Cet attribut peut prendre les valeurs :

auto (par défaut) : le navigateur décide s'il doit précharger toute la vidéo, uniquement les métadonnées ou rien du tout.

metadata : charge uniquement les métadonnées (durée, dimensions, etc.).

none : pas de préchargement. Utile si vous souhaitez éviter le gaspillage de bande passante sur votre site.

On ne peut pas forcer le préchargement de la vidéo, c'est toujours le navigateur qui décide.

Les proportions de la vidéo sont toujours conservées. Si vous définissez une largeur et une hauteur, le navigateur fera en sorte de ne pas dépasser les dimensions indiquées mais il conservera les proportions.

Exemple:

```
<video src="mavideo.mp4" controls poster="sintel.jpg" width="600"></video>
```

Les différents éléments du formulaire

Objet	Code HTML 5
Texte simple	<code><label>Texte</label></code>
Bouton simple	<code><input type="button" name="nom" value="Bouton"></code>
Bouton image	<code><input type="image" src="image/bimage.jpg"></code>
Champ de texte	<code><input type="text" name="texte" value="rien"></code>
Mot de passe	<code><input type="password" name="monpass" value="12345"></code>
Date	<code><input type="date" name="d" value="<?php echo \$today?>"></code>
Nombre	<code><input type="number" name="nombre" value="12345"></code>
Couleur	<code><input type="color" name="couleur" value=""></code>
Champ de recherche	<code><input type="search" name="" list="datalist" value=""></code>
Case à cocher	<code><input type="checkbox" name="checkbox1" value="checkbox"></code>
Groupe radio	<code><label>Choix 1</code> <code><input type="radio" name="radio1" value="radio1"></code> <code></label></code> <code><label>Choix 2</code> <code><input type="radio" name="radio1" value="radio2"></code> <code></label></code>
Zone de texte	<code><textarea name="textarea">contenu</textarea></code>
Range	<code><input type="range" min="-100" max="100" value="0" step="2"</code> <code>name="power" list="powers"></code> <code><datalist id="powers"></code> <code><option value="0"></code> <code><option value="-30"></code> <code><option value="30"></code> <code><option value="+50"></code> <code></datalist></code>
Liste de données	<code><datalist id="identifieur"></code> <code><option value="1"></code> <code><option value="2"></code> <code><option value="3"></code> <code></datalist></code>

Select	<pre> <select name="select"> <option>Alpha</option> <option>Beta</option> <option>Delta</option> </select> </pre>
Select liste	<pre> <select name="select2" size="3"> <option>Alpha</option> <option>Beta</option> <option>Delta</option> </select> </pre>
Menu	<pre> <menu type="context"> Nouveau Ouvrir Sauver </menu> </pre> Pa supporté
Toolbar	<pre> <menu type="toolbar"> <button type="button" onclick="fnew()">Nouveau</button> <button type="button" onclick="fopen()">Ouvrir</button> <button type="button" onclick="fsave()">Sauver</button> </menu> </pre>
Combobox	<pre> <input type="text" list="comboid"> <datalist id="comboid"> <option value="0"> <option value="-30"> <option value="30"> <option value="+50"> </datalist> </pre>
Fichier	<pre> <input type="file" name="file"> </pre>
Meter	<pre> <meter min=0 max=24 value=12>12 units</meter> </pre>
Progress	<pre> <progress id="prog" max=100> </pre>
Bouton d'envoi	<pre> <input type="submit" name="envoi" value="Envoyer"> </pre>

Un site qui reprend toutes les nouveautés du formulaire et vous permet de les tester dans vos navigateurs: <http://www.scriptol.fr/html5/formulaire.php>

Balise <datalist>

Exemple:

```
<input list="age" />

<datalist id="age">
  <option value="bébé">
  <option value="enfant">
  <option value="adolescent">
  <option value="adulte">
  <option value="papi ou mami">
</datalist>
```

La balise **<datalist>** spécifie une liste pré-définis d'options pour un élément <input>.

La balise **<datalist>** est utilisé pour fournir une « **saisie semi-automatique** » sur les éléments <input>. Les utilisateurs verront une liste déroulante d'options prédéfinies lors de l'entrée des données.

Utiliser l'attribut *list* de l'élément <input> pour le relier avec un élément **<datalist>**.

Compatibilité avec les navigateurs

Firefox	Firefox 4.0+
Opera	Opera 9.0+
Opera Mobile	10.0+
Internet Explorer	Internet Explorer 10+
Google Chrome	Chrome 20+

Remarque: Un navigateur ne supportant pas l'élément n'affichera aucune suggestion. Le champ <input> lié restera cependant totalement fonctionnel, l'entrée sera libre. L'exemple présenté dans cet article ne fonctionne que sur es alternatives, se rapporter au dernier point.

Alternative avec <select>

Pour les navigateurs ne supportant pas <datalist>, une alternative simple peut être trouvée en complétant le formulaire par un <select>. Le champ primaire reste libre, tandis que la liste de choix est présentée pour ces navigateurs.

Exemple:

```
<label for="www">Adresse web</label>
<input id="www" type="url" list="urldata" name="adresseweb">
<datalist id="urldata">
<label for="adresse">ou sélectionner dans la liste</label>
<select name="adresse" id="adresse">
<option value="1">http://www.coucou.com/</option>
<option value="2">http://www.bing.com/</option>
<option value="3">http://www.google.fr/</option>
<option value="4">http://www.yahoo.fr/</option>
</select>
</datalist>
```

Attributs importants pour les éléments du formulaire

HTML5 introduit de nombreuses nouveautés pour les formulaires pour améliorer l'aide à la saisie et les contrôles disponibles pour l'utilisateur. Plusieurs attributs simples à mettre en place améliorent la prise en charge des formulaires, tout en se passant de JavaScript.

1. Placeholder

Placeholder est un attribut qui permet de renseigner un texte indicatif par défaut dans un champ de formulaire.

C'est une valeur qui s'efface dès que l'utilisateur active le champ de formulaire.

Exemple:

```
<input type="text" placeholder="Entrez un pseudo">
```

2. Required

Required permet de rendre obligatoire le remplissage d'un champ et bloquer la validation du formulaire si l'un des champs (concernés par cet attribut) n'a pas été renseigné.

Exemple:

```
<input type="text" required>
ou encore
<input type="text" required=" required ">
```

3. Pattern

L'attribut **pattern** bloquent le processus d'envoi des données d'un formulaire. A l'aide d'expressions régulières vous allez contrôler le contenu inséré dans un champ.

Exemple:

```
<input type="text" pattern="[A-F][0-9]{5}">
```

Ce champ attend une valeur numérique d'au moins 5 chiffres précédés d'une lettre majuscule comprise entre A et F.
Si le format n'est pas respecté, le navigateur en informe l'utilisateur.

4. Contenteditable

Tout élément de HTML peut devenir éditable par l'internaute. Ce peut être un paragraphe <p>, une liste , etc...

Pour avoir une utilité, il faut que cela soit stocké pour une prochaine session. Cette attribut est couplé avec les fonctions localStorage, qui conservent des données dans un espace réservé au navigateur.

Remarque: pas bien supporté par les navigateurs

5. Form

C'est une balise, mais c'est aussi maintenant un attribut pour les objets de formulaire. Cela permet de les placer n'importe où dans la page et de les associer à un formulaire en indiquant son ID.

Exemple:

```
<form id="x">  
</form>  
<button form="x" />
```

6. Novalidate

Attribut de form qui indique que les données de formulaire doivent être envoyées sans validation. On ignore alors l'attribut required.

7. draggable, dropzone

Deux attributs pour toute balise indiquent le premier que l'on peut déplacer l'élément, pour le second que l'on peut le placer ici.

8. async

Attribut de script qui fait qu'un script JavaScript sera exécuté de façon asynchrone, après chargement complet de la page, même s'il se trouve en tête de page.

Exemple

```
<script async></script>
```

9. reversed

Pour une liste , affiche des nombres en ordre descendant.

10. download

Pour une URL, permet de télécharger le fichier au lieu de l'afficher.

Exemple:

```
<a href="filename" download="newfilename"></a>
```

11. autocomplete

Cet attribut de la balise form, a deux valeurs, on, si l'autocomplétion est active, off sinon. Quand il elle est activée, ce qui l'utilisateur tape dans un champ de texte peut être automatiquement complété par le navigateur en fonction de ce qu'il a déjà entré par le passé dans le même champ.

Le développeur n'a rien d'autre à incorporer.

Exemple:

```
<form action="" autocomplete="on">
```

12. dirname

Dirname=direction name, le nom de la direction du texte, de gauche à droite ou de droite à gauche, ce qui est utile pour les pays où la direction est inversée. Cette direction pour les champs textarea et input est généralement choisie par défaut par le navigateur, selon le pays où il réside, ou selon le contenu de la page. Elle vaut ltr ou rtl.

Remarque: pas bien supporté par les navigateurs

13. min, max, step

Ces attributs concernent les champs de texte de type: number, range, date, datetime, datetime-local, month, week, time. Ils limitent les valeurs que l'on peut entrer avec respectivement, une valeur de limite inférieure, une limite supérieure et un incrément.

Exemple:

```
<input type="number" min="1" max="100" step="3" />
```

14. Attributs personnalisés

N'avez-vous jamais été tenté d'employer un attribut créé de toutes pièces, non référencé dans les spécifications ? HTML 5 autorise la mise en oeuvre de tels attributs personnalisés sous certaines conditions :

- Le nom de l'attribut doit être composé d'au moins un caractère et être préfixé de la chaîne data- afin d'être valide.
- Il ne doit pas contenir de caractères en majuscules.
- La valeur de l'attribut personnalisé peut être n'importe quelle chaîne de caractères.

Balise <details>

Exemple:

```
<details>
<summary>Copyright 2012.</summary>
<p>M.Albert - Tous droits réservés</p>
</details>
```

La balise **<details>** définit les détails supplémentaires que l'utilisateur peut afficher ou masquer à la demande.

Compatibilité navigateur de l'élément <datalist>

Internet Explorer Internet Explorer 11 ne le supporte pas (tout est visible).

Balise <embed>

Exemple:

```
<embed width="480px" height="360px" src="exemples/exemple-animation-flash.swf"
type="application/x-shockwave-flash" />
```

Attributs	Définition	Valeurs
height	Hauteur de l'élément affiché	px
src	URL	URL
type	Type MIME du contenu rendu par la balise embed	
width	Largeur de l'élément affiché	px

Balise <figure>

La balise **<figure>** spécifie un contenu autonome comme des illustrations, des diagrammes, des photos, des listes de codes, etc ...

Exemple:

```
<figure>
  
  <figcaption>Photo 01 : Le logo de Marc Lambert.</figcaption>
</figure>
```

Balise <figcaption>

La balise <figcaption> définit une légende pour un élément <figure>.

Exemple:

```
<figure>
  
  <figcaption>Photo 01 : Le logo de M.Albert.</figcaption>
</figure>
```

Balise <mark>

La balise <mark> est un surligneur de texte. Il marque un passage spécifique, que l'on souhaite mettre en valeur d'une façon particulière, pour attirer l'attention.

Son style par défaut est celui d'un texte surligné en jaune (dans les navigateurs qui le reconnaissent).

Exemple de changement de l'apparence de mark en CSS

```
mark
{
  background-color: red;
  color: white;
}
```

Balise <canvas>

L'élément HTML 5 <canvas> représente une zone de dessin bitmap (« canevas ») au sein de la page, pilotée par JavaScript et capable de restituer des formes, des diagrammes et graphiques, des animations ou autres applications visuelles.

Les fonctionnalités disponibles de base sont plutôt simplistes, mais nombreuses :

- des outils de dessins et de formes (rectangles, ellipses, arcs, chemins, lignes et courbes de Bézier) ;
- des effets (remplissages, contours, ombres, dégradés linéaires et radiaux, transparences, découpages) ;
- des transformations (zooms, rotations, translations) ;
- la gestion des images (import et export) ;
- la gestion de textes et de la typographie (polices embarquées, contours, étirements).

Exemple:

Partie HTML

```
<canvas id="dessin" width="500" height="200"></canvas>
```

Partie CSS

```
canvas {outline: 1px solid green}
```

Dessignons maintenant à l'aide du JavaScript un rectangle sur une surface de 200 pixels de large pour 100 pixels de haut :

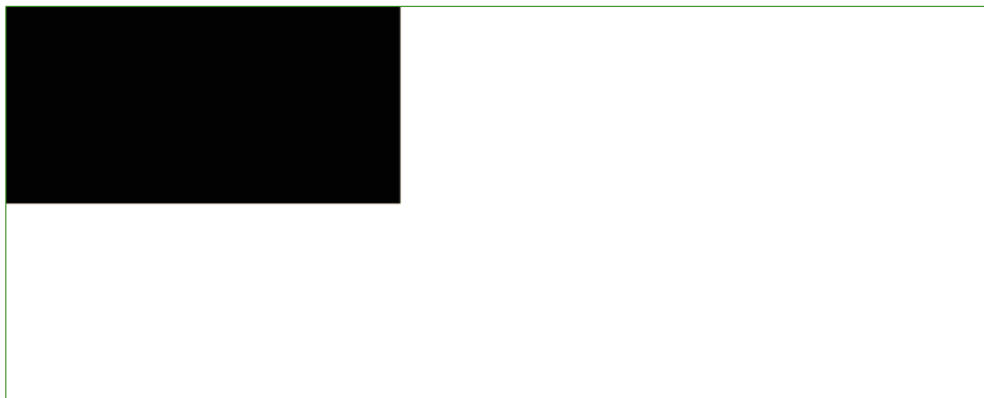
Partie JavaScript

```
<script>
var draw = document.getElementById("dessin");
var context = draw.getContext("2d");
context.fillRect(0, 0, 200, 100);
</script>
```

La première ligne de code est commune. Elle permet à JavaScript de détecter l'élément « rectangle » dans le DOM.

La deuxième ligne, obligatoire également, indique, via la méthode `getContext`, que notre canevas est de type « 2d » et non en trois dimensions.

Enfin, la troisième ligne de code applique une couleur de remplissage (noire par défaut) à un rectangle positionné en haut à gauche et occupant 200 × 100 pixels.



Balise < SVG >

Le Scalable Vector Graphics (en français « graphique vectoriel adaptable »), ou **SVG**, est un format de données conçu pour décrire des ensembles de graphiques vectoriels et basé sur XML.

SVG est un format d'image léger lorsqu'il s'agit de représenter des formes simples, car seules les informations décrivant ces formes sont stockées (coordonnées, couleurs, effets) contrairement aux images bitmap (JPG, PNG, GIF) qui doivent mémoriser le contenu pixel par pixel.

Ce principe rend les images SVG étirables sans perte de qualité. Il est également une alternative à Flash pour les petites animations lorsqu'il est combiné à SMIL, ou pour présenter des données qui doivent être dynamiques.

Inkscape est un outil gratuit et puissant qui permet de créer des images SVG.
Site web: <https://inkscape.org/fr/?lang=fr>

Editeur gratuit en ligne : <https://framavectoriel.org/svg-editor.html>

Intégrer SVG à sa page web

HTML5 introduit un nouvel élément <svg> afin d'embarquer du contenu SVG dans une page web.

```
<svg width="640" height="480" xmlns="http://www.w3.org/2000/svg"
xmlns:svg="http://www.w3.org/2000/svg">
```

```
  <rect id="svg_1" height="49" width="59" y="166" x="99.5" stroke-width="5"
stroke="#000000" fill="#FF0000"/>
```

```
</svg>
```

Introduction au langage CCS3

Les nouveaux sélecteurs CSS3

1. Les sélecteurs d'attributs

Il y a 3 nouveaux sélecteurs

[attr^="stringValue"]

Ce sélecteur permet de sélectionner un élément DOM dont l'attribut "attr" commence exactement par la valeur "stringValue".

Exemple :

```
p.example
```

```
{  
margin:0;  
padding:10px;  
color:#000;  
}
```

```
p.example[title^="ess"]
```

```
{  
color:#fff;  
background:#333;  
}
```

```
<p class="example"> je n'ai pas d'attribut title</p>
```

```
<p class="example" title="comment"> j'ai un attribut title mais il ne commence pas  
par "ess"</p>
```

```
<p class="example" title="essai"> j'ai un attribut title commençant par "ess"</p>
```

```
<p class="example" title="esson"> j'ai un attribut title commençant par "ess"  
également</p>
```

[attr\$="stringValue"]

Ce sélecteur permet de sélectionner un élément DOM dont l'attribut "attr" finit exactement par la valeur "stringValue".

Exemple :

```
p.example2
```

```
{  
margin:0;  
padding:10px;  
color:#000;  
}
```

```
p.example2[title$="sai"]
```

```
{  
color:#fff;  
background:#045FB4;  
}
```

```
<p class="example2"> je n'ai pas d'attribut title</p>
```

```
<p class="example2" title="comment"> j'ai un attribut title mais il ne commence pas  
par "ess"</p>
```

```
<p class="example2" title="essai"> j'ai un attribut title commençant par "ess"</p>
```

```
<p class="example2" title="esson"> j'ai un attribut title commençant par "ess"  
également</p>
```

[attr*="stringValue"]

Ce sélecteur permet de sélectionner un élément DOM dont l'attribut "attr" comporte au moins une fois la valeur "stringValue".

Exemple :

```
p.example3
```

```
{  
margin:0;  
padding:10px;  
color:#000;  
}
```

```
p.example3[title*="val"]
```

```
{  
color:#fff;  
background:#990000;  
}
```

```
<p class="example3"> je n'ai pas d'attribut title</p>
```

```
<p class="example3" title="comment"> j'ai un attribut title mais il ne contient pas  
"val"</p>
```

```
<p class="example3" title="val"> j'ai un attribut title contenant au moins "val"</p>
```

```
<p class="example3" title="evaluer"> j'ai un attribut title contenant au moins "val"  
également</p>
```

```
<p class="example3" title="eval"> j'ai un attribut title contenant au moins "val"  
également</p>
```

2. Le combinateur d'adjacence directe

Permet d'ajouter un style à tous les éléments qui suivent un élément particulier.

Exemple :

```
.example4 div
{
margin:0;
padding:10px
color:#000;
}
.example4 div~p
{
color:#fff;
margin:20px;
width:200px;
padding:5px;
border:1px solid #333;
background:#006644;
}
<div class="example4">
<div>je suis l'élément particulier div</div>
<p> je suis un p qui suit le div (l'élément particulier)</p>
<p>je suis un p qui suit le div (l'élément particulier)</p>
<span>je suis un span</p>
<p>je suis un p qui ne suit pas le div (l'élément particulier)</p>
</div>
```

3. Les pseudo-classes

Plusieurs pseudo-classes ont été ajoutées :

:root

Ce sélecteur représente un élément qui est la racine d'un document. Par exemple, en HTML 4, l'élément est html.

:nth-child(expression)

Ce sélecteur permet de cibler tous les éléments en se basant sur leur position dans la liste des enfants de leur parent.

`expression` peut être un nombre, une expression numérique ou un mot clé tel que "odd" ou "even".

Exemple de pseudo-classes :

```
.exempleTable
{
width:100%;
border:1px solid #444;
}

.exempleTable tr:nth-child(even)
{ //tous les enfants aux numéros pairs
background:#999999;
text-shadow: 2px 2px 5px #111;
color:#fff;
}

.exempleTable tr:nth-child(odd)
{ //tous les enfants aux numéros impairs
background:#990000;
color:#fff;
}

.exempleTable tr:nth-child(3n)
{ //tous les 3 enfants
background:#045FB4;
color:#fff;
}

.exempleTable tr:nth-child(7)
{ //l'enfant numéro 7
background:#006400;
text-shadow: 2px 2px 2px #fff;
color:#000;
}
```

```
<table class="exampleTable">
    <tr>
        <td>1ere ligne</td>
    </tr>
    <tr>
        <td>2eme ligne</td>
    </tr>
    <tr>
        <td>3eme ligne</td>
    </tr>
    <tr>
        <td>4eme ligne</td>
    </tr>
    <tr>
        <td>5ere ligne</td>
    </tr>
    <tr>
        <td>6eme ligne</td>
    </tr>
    <tr>
        <td>7eme ligne</td>
    </tr>
    <tr>
        <td>8eme ligne</td>
    </tr>
</table>
```

:nth-last-child(expression)

Ce sélecteur accepte les mêmes arguments que :nth-child() et correspond au dernier enfant d'un élément parent.

C'est le même principe que le :nth-child() sauf que l'on part de la fin.

Exemple:

```
.exampleTable2{
    width:100%;
    border:1px solid #444;
}
.exampleTable2 tr:nth-last-child(odd){ /*tous les enfants aux numéros
impairs depuis la fin.*/
    background:#990000;
    color:#fff;
}
.exampleTable2 tr:nth-last-child(-n+2){ /*les 2 derniers enfants.*/
    background:#045FB4;
    color:#fff;
}
.exampleTable2 tr:nth-last-child(7){ /*l'enfant numéro 7 en partant de la fin
donc la 2ème ligne du tableau.*/
    background:#006400;
    text-shadow: 2px 2px 2px #fff;
    color:#000;
}
```

```
<table class="exampleTable2">
  <tr>
    <td>1ere ligne</td>
  </tr>
  <tr>
    <td>2eme ligne</td>
  </tr>
  <tr>
    <td>3eme ligne</td>
  </tr>
  <tr>
    <td>4eme ligne</td>
  </tr>
  <tr>
    <td>5ere ligne</td>
  </tr>
  <tr>
    <td>6eme ligne</td>
  </tr>
  <tr>
    <td>7eme ligne</td>
  </tr>
  <tr>
    <td>8eme ligne</td>
  </tr>
</table>
```

:

:last-child

Correspond à `:nth-last-child(1)`.

:first-child

Correspond à `:nth-child(1)`.

:nth-of-type(expression)

Ce sélecteur représente un élément qui a expression frères du même type devant lui dans l'arbre DOM.

Exemple:

```
img:nth-of-type(2n+1) { float: right; }
img:nth-of-type(2n) { float: left; }
```

alternance du flottement des images

:nth-last-of-type(expression)

Ce sélecteur représente un élément qui a expression - 1 frères du même type après lui dans l'arbre DOM.

Exemple

```
body > h2:nth-of-type(n+2):nth-last-of-type(n+2)
```

représente tous les h2 fils d'un élément XHTML body sauf le premier et le dernier.

:first-of-type

Correspond à `:nth-of-type(1)`. `:first-of-type` représente un élément qui est le premier enfant de son type dans la liste des enfants de son élément parent.

:last-of-type

Correspond à `:nth-last-of-type(1)`. Représente un élément qui est le dernier enfant de son type dans la liste des enfants de son élément parent.

:only-child

Correspond à un élément qui n'a aucun frère. Cette pseudo-classe correspond à `:first-of-type:last-of-type` ou `:nth-of-type(1):nth-last-of-type(1)`.

:checked

Correspond aux éléments cochés d'un formulaire.

:empty

Correspond aux éléments n'ayant pas d'enfant ou ne contenant rien (un paragraphe vide par exemple).

:target

Représente un élément qui est la cible de l'URI.

Exemple :

```
p:target
{
background:#900
}
```

Tout élément p qui sera la cible de l'URI (via l'ID # en tant que ancre) aura pour couleur d'arrière plan la valeur "#900".

:valid

Représente un élément qui dans un formulaire est valide.

:invalid

Représente un élément qui dans un formulaire est n'est pas valide.

Les pseudo-éléments

::first-line

Applique la règle de style à la première ligne du texte de l'élément.

Exemple:

```
p::first-line
{
text-transform: uppercase
}
```

La 1ère ligne de(s) élément(s) "p" est mise en majuscule.

::first-letter

Applique la règle de style à la première lettre du texte de l'élément.

Exemple

```
p::first-letter
{
font-size:2em
}
```

La 1ère lettre de(s) élément(s) "p" a une taille de police de 2em.

::selection

Applique la règle de style à la sélection du texte de l'élément faite par l'utilisateur.

Exemple:

```
p::selection
{
background:#006644
}
```

A la sélection, le texte sélectionné aura une couleur d'arrière plan de valeur '#006644'.

Remarque:

IE sur mobile ne l'accepte pas.

Chrome ne traite pas ce pseudo-élément sur les input.

::before et ::after

Génère un contenu avant ou après un contenu d'un élément.

Ces pseudo-elements existent en CSS2.1 sous forme :before et :after.

Exemple:

```
P.note:before
{
content: "Note : "
}
```

cette règle insère la chaîne "Note : " avant le contenu de chacun des éléments P dont l'attribut "class" a la valeur "note"

Exercice: en utilisant les sélecteurs CSS3, réalisez la figure suivante:

	Origine	Prix
Kiwi	France	42€
Pastèque	Taiïwan	1337€
Papaye	Roumanie	1.3€
Melon	France	12€
Litchi	Groënland	15.50€

Prix des fruits

Propriété de style pour le texte

La taille

Pour modifier la taille du texte, on utilise la propriété CSS **font-size**.
Deux types de valeur possible: en valeur absolue ou relative.

Indiquer une taille absolue : en pixels, en centimètres ou millimètres.

Exemples:

```
p
{
  font-size: 14px; /* Paragraphes de 14 pixels */
}
h1
{
  font-size: 40px; /* Titres de 40 pixels */
}
```

Indiquer une taille relative : en pourcentage, mot clé ou « em » ...

Exemples:

```
p
{
  font-size: small;
}
h1
{
  font-size: large;
}
p
{
  font-size: 0.8em;
}
h1
{
  font-size: 1.3em;
}
```

La police

La propriété CSS qui permet d'indiquer la police à utiliser est **font-family**. Vous devez écrire le nom de la police comme ceci :

Exemple:

```
h1
{
  font-family: arial, "Arial Black", Verdana, sans-serif;
}
```

Nouveauté CSS3: la possibilité d'utiliser des polices ne se situant pas sur l'ordinateur du client avec la propriété **@font-face**

Exemple:

```
@font-face {
  font-family: 'Toons';
  src: url('Toons.eot') format('eot'),
  url('Toons.woff') format('woff'),
  url('Toons.ttf') format('truetype'),
  url('Toons.svg') format('svg');
}

h1
{
  font-family: 'Toons', Arial, serif;
}
```

Le Gras

La propriété CSS pour mettre en gras est **font-weight**

Les différentes valeurs sont exprimées par un mot clé ou par un poids.

font-weight: normal|bold|bolder|lighter|initial|inherit;

font-weight: 100|200|300|400|500|600|700|800|900|inherit;

Couper un mot avec la propriété word-wrap

Lorsqu' un mot est trop long, le comportement par défaut est de le faire déborder sans affecter la dimension du conteneur.

word-wrap force la césure d'un mot.

Les valeurs:

- word-wrap:normal
- word-wrap:break-word

Dépassement du texte avec la propriété text-overflow

text-overflow décrit le comportement en cas de dépassement du texte par rapport à son container

Cette propriété n'affecte que le contenu qui dépasse de la boîte englobante dans le sens de la progression en ligne. Par exemple, on ne parle pas ici du texte qui dépasserait vers le bas d'une boîte. Le texte peut dépasser lorsqu'on empêche le retour automatique à la ligne (par exemple avec **white-space: nowrap**) ou lorsqu'un seul mot est trop long pour tenir dans le conteneur.

Hidden : cache le contenu

Scroll : crée une scrollbar

Auto : choisi le comportement en fonction du contenu

text-overflow associé à la propriété ellipsis ajoute des points de suspension quand le contenu est caché

Les ombres sur le texte

La propriété **text-shadow** permet d'appliquer de l'ombre au texte.

Exemple:

```
h1
{
text-shadow: 2px 2px 2px #ff0000;
color: #000;
}
```

Soulignement et "décorations"

La propriété CSS associée est : **text-decoration**.

Voici les différentes valeurs qu'elle peut prendre :

underline : souligné.

line-through : barré.

overline : ligne au-dessus.

blink : clignotant. Ne fonctionne pas ou plus sur tous les navigateurs.

none : normal (par défaut).

Exemples:

```
h1
{
text-decoration: blink;
}

.souligne
{
text-decoration: underline;
}

.barre
{
text-decoration: line-through;
}

.ligne_dessus
{
text-decoration: overline;
}
```

L'alignement du texte

la propriété **text-align** nous permet de spécifier l'alignement désiré :

Les valeurs:

left : le texte sera aligné à gauche (c'est le réglage par défaut).

center : le texte sera centré.

right : le texte sera aligné à droite.

justify : le texte sera « justifié ». Justifier le texte permet de faire en sorte qu'il prenne toute la largeur possible sans laisser d'espace blanc à la fin des lignes.

Exemple:

```
h1
{
  text-align: center;
}
p
{
  text-align: justify;
}
```

Remarque à propos de l'alignement du **texte** par rapport à une **image**.

Pour obtenir comme en HTML un alignement d'une image par rapport au texte, par exemple que l'image soit incrustée à gauche dans un block de texte il faut faire flotter l'image avec la propriété **float**.

Les valeurs sont

left : le texte se poursuit en-dessous après un float: left;

right : le texte se poursuit en-dessous après un float: right;

both : le texte se poursuit en-dessous, que ce soit après un float: left; ou après un float: right.

Columns

CSS 3 offre la possibilité de présenter un texte sur plusieurs colonnes, à l'instar des journaux papier, via `column` et ses propriétés dérivées.

Ces différentes propriétés peuvent être scindées en trois parties fonctionnelles :

- Le premier groupe de propriétés définit le nombre et la taille des colonnes :
 - `columns` (raccourci de `column-count` et `column-width`) : nombre de colonnes et éventuellement largeur de chaque colonne ;
 - `column-min-width` : largeur minimale de chaque colonne ;
 - `column-width-policy` : le mode d'affichage des colonnes (valeurs "flexible", "strict" ou "inherit").
- Le second groupe gère ce qu'il y a entre les colonnes :
 - `column-gap` : distance entre chaque colonne ;
 - `column-rule` (raccourci de `column-rule-color`, `column-rule-style` et `column-rule-width`) : couleur, style et largeur de la séparation entre colonnes.
- Enfin, une propriété `column-span` permet à un élément de s'étendre sur plusieurs colonnes.

Compatibilité columns

Le positionnement multicolonne est - contre toute attente - plutôt bien supporté par le collège des navigateurs. Les anciennes versions d'Internet Explorer faisant bien entendu exception à la règle.

En pratique, il est actuellement utilisable sans problème en production sur les navigateurs mobiles que l'on retrouve sur smartphones et tablettes.

La transparence

La propriété utilisée est **opacity** qui indique le niveau de transparence.

Les valeurs sont comprises entre 0 et 1

Exemple:

```
div
{
opacity: 0.5;
}
```

Un inconvénient fréquemment rencontré est que **opacity** s'applique à l'élément dans son intégralité, y compris tous ses descendants, ce qui n'est pas sans poser des problèmes si l'on désire que l'un des enfants ne bénéficie pas de ce lourd héritage : en pratique, et sans jouer avec des positionnements hors flux complexes, il est impossible d'annuler l'opacité de son ancêtre.

Compatibilité:

On peut aussi jouer sur la **transparence** d'un arrière plan de couleur

La notation est RGBA

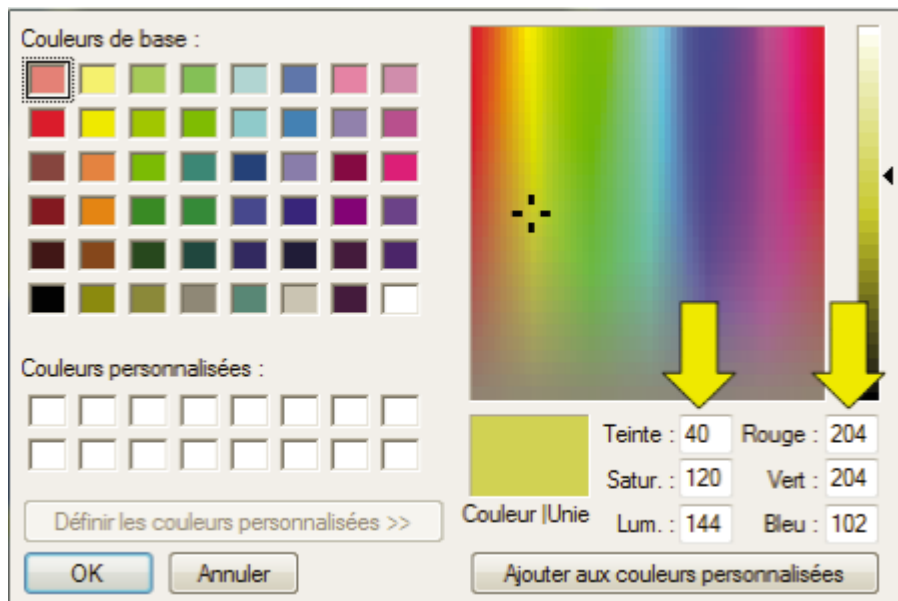
Le **a** ajoute le niveau de transparence et prend des valeurs situées entre 0 et 1.

Exemple:

```
p
{
background-color: rgb(255,0,255,0.5);
}
```

La notation HSLa

La notation **HSLa** traduit les couleurs via des canaux de teinte, de saturation, de luminosité et de transparence :



- H (Hue) : nuance de couleur entre 0 et 360 (0 correspond au rouge, 120 au vert et 240 au bleu) ;
- S (Saturation) : valeur en pourcentage (0 est complètement terne, 100 % est au maximum de la vibrance) ;
- L (Luminosity) : valeur de 0 (sombre) à 100 % (clair) ;
- a (alpha) : degré de transparence, valeur de 0 à 1.

Compatibilité des notations RGBa et HSLa

Ces systèmes de notation sont pris en compte sur Chrome, Safari, Opera, Firefox 4, et Internet Explorer 9

Réaliser des coins arrondis

Propriétés utilisées :

border

border-radius

La propriété border-radius peut accepter 4 valeurs pour l'arrondissement de chaque coins. La 1ère valeur correspond au coin haut gauche, puis on tourne dans le sens des aiguilles d'une montre.

On peut n'en indiquer que 2, qui correspondront aux coins opposés, ou une seule pour un même arrondis sur les 4 coins.

Exemple:

```
.coin
{
background-color:#E4EFFF;
border:1px solid #9FC6FF;
padding:5px;
/*arrondir les coins en haut à gauche et en bas à droite*/
-moz-border-radius:10px 0;
-webkit-border-radius:10px 0;
border-radius:10px 0;
}
```

Créer des ombres sur des boîtes

La propriété box-shadow doit recevoir plusieurs valeurs (les deux premières étant obligatoires) :

Une valeur de décalage horizontal

Une valeur de décalage vertical

Une valeur pour l'effet flou

Une valeur l'étendue de l'ombre

La couleur de l'ombre.

Exemple:

```
.ombre1
{
background-color:#C0C0C0;
border:1px solid gray;
padding:5px;
box-shadow:2px 2px 10px gray;
-moz-box-shadow:2px 2px 10px gray;
-webkit-box-shadow:2px 2px 10px gray;
}
```

On peut aussi rajouter la valeur inset qui va ombrer l'intérieur en donnant un effet de relief en creux.

```
.ombre2 {
background-color:#C0C0C0;
border:1px solid gray;
padding:5px;
box-shadow:2px 2px 10px gray inset;
-moz-box-shadow:2px 2px 10px gray inset;
-webkit-box-shadow:2px 2px 10px gray inset;
}
```

Remarque: on peut déclarer plusieurs ombres à un élément.

```
.ombre3
{
background-color:#C0C0C0;
padding:5px;
box-shadow:2px 2px 2px gray,
-1px -1px 2px white;
}
```

Arrière-plans multiples

CSS 3 rend possible l'affichage de plusieurs images d'arrière-plan sur un même élément en cumulant les valeurs au sein des propriétés `background-image`, `background-position` et `backgroundrepeat`, ces valeurs étant simplement séparées par une virgule.

Le résultat est similaire à des calques d'un logiciel graphique tel que Photoshop : la première image déclarée dans la liste sera au premier plan. Si une couleur de fond est déclarée, elle sera toujours reléguée au dernier plan.

L'ordre de déclaration est important : dans l'exemple ci-après, la position `left top` s'applique uniquement à la première image et `right bottom` s'applique uniquement à la deuxième image. Si une seule propriété est spécifiée (dans notre exemple `no-repeat`), elle sera appliquée à l'ensemble des images.

Exemple:

```
div
{
background-image: url("first.jpg"),url("second.jpg");
background-position: left top,right bottom;
background-repeat: no-repeat;
}
```

Ou, en employant la propriété raccourcie `background` :

```
div {
background: url("first.jpg") left top no-repeat, url("second.jpg") right bottom
no-repeat;
}
```

background-position admet des valeurs absolues pour le positionnement

Exemple:

```
div
{
background-image: url("image.jpg");
background-position: 50px 30px
}
```

ou encore

```
div
{
background-image: url("image.jpg");
background-position: right 20px left 10 px;
}
```

Modifier la taille d'un arrière plan

CSS 3, via la propriété `background-size`, offre un moyen de spécifier les dimensions des images d'arrière-plan dans le but de les adapter à celles de l'élément sur lequel elles sont appliquées.

Il suffit d'indiquer une valeur ou deux (horizontale et verticale) en pixels ou en pourcentage.

Ces valeurs sont relatives aux dimensions du bloc, c'est-à-dire par défaut ses composantes de contenu et de marges internes

L'exemple ci-après redimensionne l'arrière-plan de manière à ce qu'il recouvre exactement tout l'élément `<div>`:

```
div {
width: 400px;
height: 400px;
font-size: 3em;
color: white;
background: #6B9A49 url(image.jpg) left top no-repeat;
background-size: 100% 100%;
}
```

Il est possible d'indiquer la valeur `auto` pour conserver le ratio visuel de l'image. Ainsi, dans l'exemple suivant, toutes les occurrences de l'image vont occuper chacune exactement 50 % de la largeur du bloc :

```
div {
background: #6B9A49 url(background.png) left top repeat;
background-size: 50% auto;
}
```

De manière plus pratique **`background-size`** admet dorénavant de nouvelles valeurs: **`contain`** et **`cover`**

`Contain`: redimensionne l'image d'arrière plan en conservant le ratio largeur/hauteur de manière à ce qu'elle soit entièrement visible

`Cover`: redimensionne l'image d'arrière plan de manière à tout couvrir quitte à ce qu'elle ne soit pas entièrement visible.

Les filtres CSS3

Fonctionnalité qui peut être utilisée à la fois sur des éléments HTML et des images, mais qui a plus d'intérêt à être utilisée sur des images comme pour la création d'une galerie par exemple.

En utilisant uniquement le CSS3 vous allez être capable de créer tous les effets suivants :

- greyscale ;
- blur ;
- saturate ;
- sepia ;
- hue rotate ;
- invert ;
- brightness ;
- contrast ;
- opacity.

Exemple

```
img
{
    filter: type(value);
    -webkit-filter: type(value);
    -moz-filter: type(value);
    -ms-filter: type(value);
    -o-filter: type(value);
}
```

Exemple pour l'effet **échelle de gris**

nous allons définir une valeur pour un effet noir et blanc total puis au survol de l'image par la souris nous allons retirer l'effet.

```
<section class="grayscale">
<h2>CSS Greyscale</h2>
  
</section>
```

```
.grayscale img
{
  filter: grayscale(1);
  -webkit-filter: grayscale(1);
  -moz-filter: grayscale(1);
  -o-filter: grayscale(1);
  -ms-filter: grayscale(1);
}
.grayscale img:hover
{
  filter: grayscale(0);
  -webkit-filter: grayscale(0);
  -moz-filter: grayscale(0);
  -o-filter: grayscale(0);
  -ms-filter: grayscale(0);
}
```

Remarque: très mal supporté par les navigateurs, pas IE (pc), pas Firefox, pas Opera Mini

La fonction `calc()` en CSS3

calc() est une fonction CSS3 offrant la possibilité de réaliser des opérations mathématiques en CSS (addition, soustraction, division et multiplication).

Compatibilité

La compatibilité de cette fonction est encore un peu limitée aux navigateurs récents. Les gros problème étant les versions Internet Explorer antérieures à IE9, et les anciens Android (avant 4.4).

Exemple:

nav, section

```
{  
float:left  
}
```

nav

```
{  
width: 250px;  
}
```

section

```
{  
width: calc(100% - 250px)  
}
```

A propos des unités de mesure

Le W3C nous explique

EM, PX, PT, CM, IN...

CSS offre différentes unités pour exprimer les dimensions. Certaines proviennent de la typographie, comme le point (**pt**) et le pica (**pc**), d'autres sont connues pour leur usage quotidien, comme le centimètre (**cm**) et le pouce (**in**). Et il y a également une unité "magique" inventée spécifiquement pour CSS: le pixel **px**. Est-ce que cela signifie que différentes propriétés nécessitent différentes unités ?

Non, les unités n'ont rien à voir avec les propriétés, mais avec le média de sortie: écran ou papier.

Il n'y a pas de restriction à utiliser telle unité à tel ou tel endroit. Si une propriété accepte une valeur en **px** ('margin: 5px') elle accepte également une valeur en pouces ou en centimètres ('margin: 1.2in; margin: 0.5cm') et vice-versa.

Mais généralement, vous utilisez un ensemble différent d'unités selon que vous affichez sur un écran ou que vous imprimez sur du papier. La table ci-dessous recommande les différents usages:

	Recommandé	Usage occasionnel use	Non recommandé
Écran	em, px, %	ex	pt, cm, mm, in, pc
Imprimante	em, cm, mm, in, pt, pc, %	px, ex	

La relation entre les unités absolues est la suivante: 1in = 2.54cm = 25.4mm = 72pt = 6pc

Si vous avez un double-décimètre, vous pouvez vérifier la précision de votre support de sortie: voici une boîte de 1in (2.54cm) de hauteur:

↑
72pt
↓

Les unités appelées *absolues* (**cm**, **mm**, **in**, **pt** and **pc**) signifient la même chose en CSS que partout ailleurs, *mais uniquement si votre support de sortie a une résolution assez élevée*. Sur une imprimante laser, 1cm devrait correspondre exactement à 1 centimètre. Mais pour des supports avec une basse résolution, comme les écrans d'ordinateurs, CSS n'exige pas cela. Et effectivement, le résultat tend à être différent d'un support à l'autre et d'une implémentation de CSS à l'autre. Il est préférable de réserver ces unités pour des supports haute-résolution et en particulier pour les supports d'impression. Sur les écrans d'ordinateurs ou de portables, vous n'obtiendrez probablement pas ce que vous attendez.

Auparavant, CSS exigeait que les implementations affichent les unités absolues correctement, y compris sur les écrans d'ordinateurs. Mais comme le nombre d'implementations incorrectes dépassait le nombre d'implementations correctes et que cela n'allait pas en s'améliorant, CSS a abandonné cette condition en 2011. Aujourd'hui, les unités absolues doivent fonctionner correctement sur les sorties imprimées et sur les supports haute-résolution uniquement.

CSS ne définit pas ce que "haute-résolution" signifie. Mais comme actuellement les imprimantes d'entrée de gamme commencent à 300 dpi et que les écrans haut de gamme atteignent 200 dpi, la limite est probablement entre les 2.

Il y a une autre raison de ne pas employer les unités absolues pour des utilisations autres que l'impression: vous regardez des écrans différents à des distances différentes. 1cm sur un écran d'une station de travail apparaît petit. Mais le même sur un téléphone mobile juste en face de vos yeux apparaît grand. Il est préférable d'utiliser des unités relatives à la place, comme **em**.

Les unités **em** et **ex** dépendent de la police de caractères et peuvent être différentes pour chaque élément du document. L'unité **em** est simplement la taille de la police de caractères. Pour un élément dont la police est de taille 2in, 1em signifie donc 2in. Exprimer des tailles, comme les *margins* et les *padding*s, en **em** signifie qu'elles sont relatives à la taille de la police, et si l'utilisateur a une large police de caractères (sur un grand écran par exemple) ou au contraire une petite police (sur un smartphone),

les tailles seront proportionnelles. Les déclarations telles que **text-indent: 1.5em** et **margin: 1em** sont très courantes en CSS.

L'unité **ex** est rarement utilisée. Son but est d'exprimer des tailles relatives à l'*x-height* de la police. Le *x-height* est, schématiquement, la hauteur des plus petites lettres comme le *a*, *c*, *m*, or *o*. Les polices qui ont la même taille (donc le même **em**) peuvent varier considérablement concernant la taille de leurs plus petites lettres, et lorsqu'il est important que certaines images par exemple aient le même *x-height*, l'unité **ex** est disponible.

L'unité **px** est l'unité magique en CSS. Elle n'est pas relative à la police courante ni relative aux unités absolues. L'unité **px** est définie pour être 'petite mais visible', et de telle façon qu'une ligne horizontale de 1px de largeur peut être affichée de façon nette, sans 'arête' (ni anti-aliasing). Ce qui est net, petit et visible dépend du support et de la façon dont il est utilisé: est-ce que vous le tenez près des yeux, comme un téléphone mobile, à la distance d'un bras, comme un écran d'ordinateur, ou à une distance intermédiaire, comme une liseuse? Le **px** n'est donc pas défini comme une longueur constante, mais comme quelque chose qui dépend du type de matériel et de son usage spécifique.

Pour avoir une idée de l'apparence du **px**, prenez un écran cathodique des années 90: le plus petit point qu'il peut afficher mesure environ 1/100e de pouce (0.25mm) ou un peu plus. L'unité **px** tire son nom de ces pixels d'écrans.

De nos jours, il y a des supports qui peuvent en principe afficher des points nets plus petits (bien que vous puissiez avoir besoin d'une loupe pour les voir). Mais les documents du siècle dernier qui utilisaient des **px** dans CSS sont rendus de la même façon, quelque soit le support. Les imprimantes en particulier peuvent afficher des lignes très nettes avec des détails bien inférieurs à 1px, mais même sur les imprimantes, une ligne de 1px est rendue de la même façon qu'elle l'aurait été sur un écran d'ordinateur. Les supports changent, mais le **px** a toujours la même apparence visuelle.

En fait, CSS requiert que **1px** soit exactement 1/96e de pouce sur toutes les sortie imprimées. CSS considère que les imprimantes, contrairement aux écrans, n'ont pas besoin d'avoir différentes tailles pour **px** pour être en mesure d'imprimer des lignes nettes. Dans un média imprimé, un **px** a donc non seulement la même apparence visuelle d'un média à un autre, mais il est en plus de la même taille (de la même manière que pour **cm**, **pt**, **mm**, **in** et **pc**, comme expliqué [ci-dessus](#)).

CSS définit également que les images matricielles (comme les photos) sont affichées par défaut avec 1 pixel d'image correspondant à 1px. Ainsi, une photo avec une résolution 600 x 400 aura 600px de large et 400px de haut. Les pixels de la photo ne correspondant donc pas aux pixels du support de sortie (lequel peut être très petit), mais correspondent aux unités **px**. Cela rend possible le fait d'aligner exactement des images avec d'autres éléments du document, tant que vous utilisez l'unité **px** dans votre feuille de style, et non **pt**, **cm**, etc.

*UTILISER **EM** OU **PX** POUR LES TAILLES DES POLICES*

CSS a héritée de la typographie les unités **pt** (point) et **pc** (pica). Les imprimantes les ont traditionnellement utilisées (ainsi que les unités similaires) de préférence à **cm** ou **in**. Dans CSS il n'y a aucune raison d'utiliser **pt**, utilisez l'unité que vous préférez. Mais il y a une bonne raison de ne pas utiliser **pt** ni aucune autre unité absolue et d'utiliser seulement **em** et **px**.

Voici quelques lignes de différentes épaisseurs. Toutes ou partie peuvent être nettes (mais au minimum les lignes 1px et 2px doivent être visibles et nettes):

Si les 4 premières lignes apparaissent identiques (ou si la ligne 0.5pt n'est pas affichée), vous avez probablement un écran qui ne peut afficher des points inférieurs à 1px. Si les lignes semblent de plus en plus épaisses, vous regardez probablement cette page sur un écran en haute résolution ou sur du papier. Et si 1pt semble plus épais que 1.5px, vous avez probablement un smartphone.

L'unité magique en CSS, le **px**, est souvent très adaptée, en particulier si le style nécessite l'alignement d'un texte sur des images, ou tout simplement parce que tout ce qui a une largeur de 1px ou d'un multiple de 1px est assuré d'avoir un rendu net.

Mais pour les tailles de police, il est préférable d'utiliser **em**. L'idée est (1) de ne pas spécifier la taille de la police de l'élément BODY (en HTML), mais d'utiliser la taille par défaut du support, parce que c'est une taille que l'utilisateur peut lire confortablement; (2) exprimer les tailles de police des autres éléments en **em**: 'H1 {font-size: 2.5em}' pour rendre le H1 2½ fois plus grand que le contenu normal du corps du document.

Le seul endroit où vous pourriez utiliser **pt** (ou **cm** ou **in**) pour définir une taille de police est dans une feuille de style pour l'impression, si vous voulez être sûr que la

police imprimée aura exactement une certaine taille. Mais même dans ce cas, utiliser la taille par défaut de la police est généralement plus indiqué.

L'unité **px** vous évite donc à devoir connaître la résolution du support. Que le sortie soit en 96 dpi, 100 dpi, 220 dpi ou 1800 dpi, une dimension exprimée avec un nombre entier de **px** sera toujours rendue correctement et de façon très similaire sur tous les supports. Mais qu'en est-il si vous voulez vraiment connaître la résolution du support, par exemple, savoir si on peut utiliser une ligne en **0.5px** ? La réponse consiste à vérifier la résolution avec [Media Queries](#). Expliquer Media Queries sort du cadre de cet article, néanmoins voici un petit exemple:

```
div.mybox { border: 2px solid }
@media (min-resolution: 2dppx) {
  /* Media with 2 or more dots per px */
  div.mybox { border: 1.5px solid }
}
```

NOUVELLES UNITÉS EN CSS

Pour rendre encore plus facile le fait d'écrire des règles de style qui ne dépendent que de la taille de la police par défaut, CSS a introduit une nouvelle unité depuis 2013: Le **rem**. Le **rem** (pour "root em") est la taille de l'élément racine du document dans la police de caractères. Contrairement au **em**, qui peut être différent pour chaque élément, le **rem** est constant tout au long du document. Par exemple, pour définir la même marge gauche pour les éléments P et H1, comparez cette feuille de style antérieure à 2013:

```
p { margin-left: 1em }
h1 { font-size: 3em; margin-left: 0.333em }
```

et la nouvelle version:

```
p { margin-left: 1rem }
h1 { font-size: 3em; margin-left: 1rem }
```

D'autres nouvelles unités rendent possible le fait de spécifier des tailles relatives à la fenêtre du lecteur, ce sont les unités **vw** and **vh**. Le **vw** équivaut à 1/100e de la largeur de la fenêtre et le **vh** équivaut à 1/100e de sa hauteur. Il y a également **vmin**, qui sélectionne la plus petite valeur entre **vw** et **vh**. Et **vmax**. (vous devinerez ce qu'elle signifie.)

Comme elles sont très récentes, elles ne fonctionnent pas encore partout. Mais en ce début 2016, plusieurs navigateurs les supportent déjà.

.

Réaliser une transition

Le principe de base d'une transition CSS3 est de permettre une transition douce entre l'ancienne valeur et la nouvelle valeur d'une propriété CSS lorsqu'un événement est déclenché.

Un changement de couleur de fond au survol de la souris, ou le changement de police de caractère, se fait par défaut de façon binaire : on passe du tout au rien. À l'aide des transitions, on peut faire en sorte que ce comportement se fasse de façon progressive.

Précédemment, ce genre de comportement n'était possible qu'avec l'usage de JavaScript. Avec les CSS3 nous pouvons dorénavant utiliser exclusivement des feuilles de style.

Les propriétés spécifiques pour les transitions en CSS 3.

transition-property précise les propriétés CSS à transformer

transition-duration précise la durée de la transition

transition-timing-function précise la fonction de transition à utiliser, le modèle d'interpolation (accélération, décélération...)

transition-delay précise le retard (ou l'avance) du départ de la transition

La propriété transition-property

Cette propriété, comme son nom l'indique, permet de définir quelles propriétés seront affectées par les transitions. Ces propriétés sont précisées en les listant, séparées par des virgules.

La propriété transition-duration

Cette propriété permet de préciser la durée de la transition. Si plusieurs propriétés ont été précisées à l'aide de la propriété précédente, il est possible de préciser plusieurs valeurs pour cette propriété en les séparant également d'une virgule.

Exemple:

```
h2
{
transition-duration: 5s;
}
```

La propriété transition-timing-function

Pour calculer les valeurs intermédiaires, le navigateur procède à une interpolation. La fonction calculant la valeur de cette interpolation influence directement la fluidité de l'animation.

Certaines animations sont plus rapides sur le début et ralentissent sur la fin, d'autres sont constantes dans leur vitesse (linéaires). Afin d'exprimer ces différents modèles d'animations, les spécifications introduisent une fonction de timing. Cette fonction de timing va traduire les nombres d'entrée (exprimés entre 0 et 1) en d'autres nombres

(toujours compris entre 0 et 1). Cette traduction donnera différents rythmes à l'animation (par exemple d'accélération ou de décélération).

Les fonctions prédéfinies sont les suivantes :

ease : Rapide sur le début et ralenti sur la fin.

linear : La vitesse est constante sur toute la durée de l'animation.

ease-in : Lent sur le début et accélère de plus en plus vers la fin.

ease-out : Rapide sur le début et décélère sur la fin.

ease-in-out : Le départ et la fin sont lents.

La propriété transition-delay

La transition commence, par défaut, dès que la propriété est changée suite à l'événement. La propriété transition-delay permet d'adapter ce comportement en retardant ou en avançant le début de la transition.

La notation raccourcie transition

Tout comme pour d'autres propriétés CSS, il existe une notation raccourcie pour déclarer les transitions. Cette notation permet de décrire facilement et de manière concise les différentes propriétés en jeu à l'aide d'une seule propriété.

Exemple:

```
.box
{
width:100px;
height: 100px;
transition: width 2s ease, height 3s linear;
}

.box:hover
{
width:200px;
height: 200px;
}
```

Liste des propriétés CSS transformables

background-color
background-position
border-bottom-color
border-bottom-width
border-left-color
border-left-width
border-right-color
border-right-width
border-spacing
border-top-color
border-top-width
bottom
clip
color
font-size
font-weight

**Liste des propriétés CSS
transformables**

background-color
background-position
border-bottom-color
border-bottom-width
height
left
letter-spacing
line-height
margin-bottom
margin-left
margin-right
margin-top
max-height
max-width
min-height
min-width
opacity
outline-color
outline-width
padding-bottom
padding-left
padding-right
padding-top
right
text-indent
text-shadow
top
vertical-align
visibility
width
word-spacing
z-index

Les transformations

La propriété **transform** permet d'appliquer des transformations en deux dimensions sur un élément html.

- 1- rotation: **rotate**
- 2- translation: **translate**
- 3- redimensionnement: **scale**
- 4- distorsion: **skew**

1- La rotation

La rotation est possible via la propriété transform prenant pour valeur rotate(x). L'argument de rotate correspond à la valeur de l'angle de rotation et peut être négatif.

```
-webkit-transform: rotate(45deg);
-moz-transform: rotate(45deg);
-o-transform: rotate(45deg);
transform: rotate(45deg);
```

Attention : Sous les navigateurs Webkit (Chrome et Safari), on ne peut pas appliquer la transformation à n'importe quel tag HTML (par exemple le span ne permet pas de transformation de type rotation).

2- La translation

La valeur translate de la propriété transform permet de déplacer un élément en x et y par rapport à sa position d'origine. Cette valeur peut prendre un ou deux arguments.

Le premier argument correspond à une translation en x (horizontale) et le deuxième argument à une translation en y (verticale).

```
-webkit-transform: translate(20px, 10px);
-moz-transform: translate(20px, 10px);
-o-transform: translate(20px, 10px);
transform: translate(20px, 10px);
/*translation horizontale de 20px et translation verticale de 10px*/
```

```
-webkit-transform: translate(20px);
-moz-transform: translate(20px);
-o-transform: translate(20px);
transform: translate(20px);
/*translation horizontale et verticale de 20px*/
Ces arguments peuvent être négatifs.
```

Tout comme scale et skew, translate peut être utilisé sous cette forme :

```
transform: translateY(20px) translateX(10px);
```

3- Le redimensionnement

La valeur scale de la propriété transform permet de dilater/redimensionner un élément. Cette valeur peut prendre un ou deux arguments : scale(x) ou scale(x, y).

Avec une seule valeur, le x correspond au coefficient de dilatation en largeur (width) et en hauteur (height). Avec deux valeurs, x correspond à la largeur (width) et y à la hauteur (height).

```
-webkit-transform: scale(2);  
-moz-transform: scale(2);  
-o-transform: scale(2);  
transform: scale(2);
```

ou

```
-webkit-transform: scale(2, 1);  
-moz-transform: scale(2, 1);  
-o-transform: scale(2, 1);  
transform: scale(2, 1);
```

Il est à noter que l'on peut utiliser les sous-valeurs de scale : scaleX et scaleY.

Exemple:

```
transform: scaleX(2) scaleY(1)
```

La distorsion

La valeur **skew** de la propriété transform permet de tordre un élément. Cette valeur peut prendre un ou deux arguments (unité en degré).

S'il n'y a qu'un argument, cela équivaut à une distorsion horizontale. S'il y a le deuxième argument, celui-ci contrôlera la distorsion verticale.

glissement horizontal :

```
-webkit-transform: skew(30deg);  
-moz-transform: skew(30deg);  
-o-transform: skew(30deg);  
transform: skew(30deg);
```

glissement horizontal et vertical :

```
-webkit-transform: skew(30deg, 20deg);  
-moz-transform: skew(30deg, 20deg);  
-o-transform: skew(30deg, 20deg);  
transform: skew(30deg, 20deg);
```

Remarque: les arguments de la valeur skew peuvent être négatifs :

```
-webkit-transform: skew(30deg, -15deg);  
-moz-transform: skew(30deg, -15deg);  
-o-transform: skew(30deg, -15deg);  
transform: skew(30deg, -15deg);
```

Les animations

Les **Animations** permettent de modifier les valeurs de propriétés CSS plusieurs fois dans le temps, tel un scénario Flash. C'est la possibilité d'utiliser plusieurs.

Déclarer l'animation

Il faut pour cela utiliser `@keyframes`, lui donner un nom, puis gérer les "étapes" en pourcentage à l'intérieur de cette déclaration. Les mots-clés `from` et `to` correspondent respectivement à 0 et 100%.

Remarque : Il est à noter que les keyframes 0% et 100% sont obligatoires et que leur omission empêchera le fonctionnement de votre animation sur certains navigateurs.:

```
@keyframes nomAnim
{
  0% { color:blue; }
  35% { color:green; }
  100% { color:red; }
}
```

Les propriétés

Les animations se décomposent en plusieurs **propriétés**, décrites ci-dessous:

- **animation-name** : le nom de l'animation à utiliser.
- **animation-duration** : le temps total de l'animation.
- **animation-timing-function** : la méthode d'interpolation (accélération, décélération).
Voici quelques valeurs possibles (voir explications plus bas) : `linear`, `ease`, `ease-in`, `ease-out`, `ease-in-out`, `steps` (nombre, `start` | `end`), `cubic-bezier`(`x1`, `y1`, `x2`, `y2`)
- **animation-iteration-count** : le nombre de répétition de l'animation. La valeur infinie permet de jouer une animation en continu.
- **animation-direction** : permet de jouer une animation en sens inverse (en fonction du cycle).
- **animation-play-state** : mettre en pause l'animation. En cours d'étude de suppression par le W3C.
- **animation-delay** : le temps avant que l'animation ne démarre.
- **animation-fill-mode** : conserver l'état de l'animation avant le début où après la fin de celle-ci.

Il existe aussi la propriété animation raccourcie:

Voici un exemple de la propriété **animation raccourcie**.

Cette animation durera 5 secondes avec un délai de 2 secondes et utilise une méthode `linear`, c'est à dire sans accélération, ni décélération.

Exemple:

```
div
{
  animation: nomAnim 5s linear 2s;
}
```

Compatibilité dans les navigateurs

Les Animations CSS3 sont compatibles avec IE 10, Firefox 5, Chrome 4, Safari 4, Opéra 12 et sur iOS et Android 4. Cependant, pour pouvoir les utiliser, il faut préfixer toutes les propriétés ainsi que les étapes définies avec @keyframes comme suit:

```
@-webkit-keyframes nomAnim{ ... }
@-moz-keyframes nomAnim{ ... }
@-ms-keyframes nomAnim{ ... }
@-o-keyframes nomAnim{ ... }
@keyframes nomAnim{ ... }

div
{
  -webkit-animation: nomAnim 5s linear 2s;
  -moz-animation: nomAnim 5s linear 2s;
  -ms-animation: nomAnim 5s linear 2s;
  -o-animation: nomAnim 5s linear 2s;
  animation: nomAnim 5s linear 2s;
}
```

Remarque:

- IE10, Firefox 16 et Opera 12.1 supportent les propriétés non-préfixées.
- Site de préfixage automatique: <http://pleeease.io/play/>

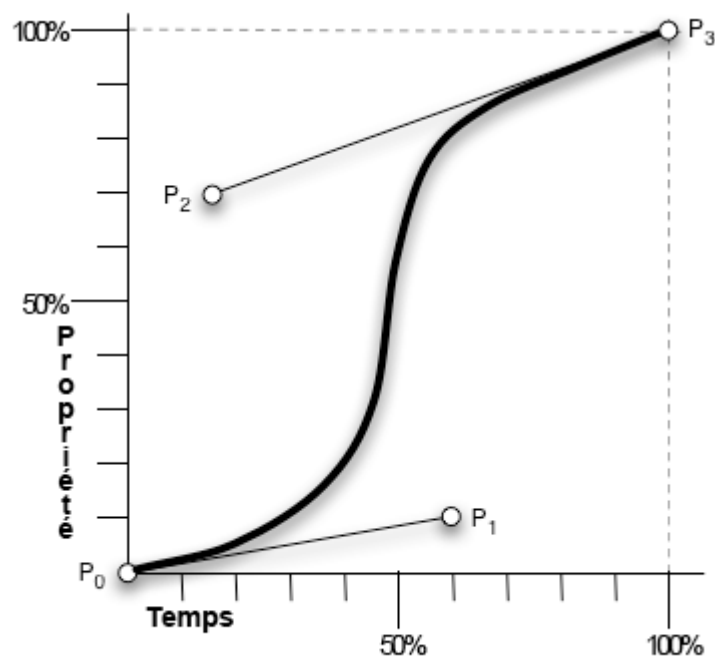
Fonctionnement des fonctions de timing - transition-timing-function (alsacreation.com)

Comme nous l'avons vu précédemment, autant les transitions que les animations ont recours à des effets de timing. Pour rappel, ces effets sont utilisés en tant que valeur des propriétés suivantes:

- transition-timing-function
- animation-timing-function (dans un bloc de code CSS ou au sein d'une keyframe)

Ces effets de timing permettent de **gérer la fluidité d'une transition** (entre deux propriétés CSS ou entre deux keyframes). Un effet de timing peut-être représenté sous deux formes: une fonction en escalier (stepping function), ou une courbe de Bézier cubique.

Dans le cas de ces deux fonctions, on pourra se les représenter sur un graphique où l'axe des Y représentera chaque étape entre la valeur initiale et la valeur finale de la propriété CSS subissant une transition et l'axe des X représentera l'avancement dans le temps de la transition (de son début à sa fin).



Dans le cas du graphique précédent (où une courbe de Bézier est utilisée), notre animation commencera lentement et finira lentement.

De base, les navigateurs fournissent quelques fonctions de transitions par défaut. Celles-ci sont:

- ease: décélère légèrement au début et termine sa transition lentement. (cubic-bezier(0.25, 0.1, 0.25, 1.0))
- linear: la transition sera stable sans décélération ni accélération. (cubic-bezier(0.0, 0.0, 1.0, 1.0))
- ease-in: commencera lentement (cubic-bezier(0.42, 0, 1.0, 1.0))
- ease-out: finira lentement (cubic-bezier(0, 0, 0.58, 1.0))
- ease-in-out: commencera et finira lentement (cubic-bezier(0.42, 0, 0.58, 1.0))
- step-start: L'élément prendra la valeur finale de sa transition dès le déclenchement de la transition. Autrement dit, il n'y aura pas de transition visible. (steps(1, start))
- step-end: L'élément prendra la valeur finale de sa transition à la fin de la durée prévue par celle-ci. Autrement dit, on aura l'impression qu'un délai a eu lieu. (steps(1, end))

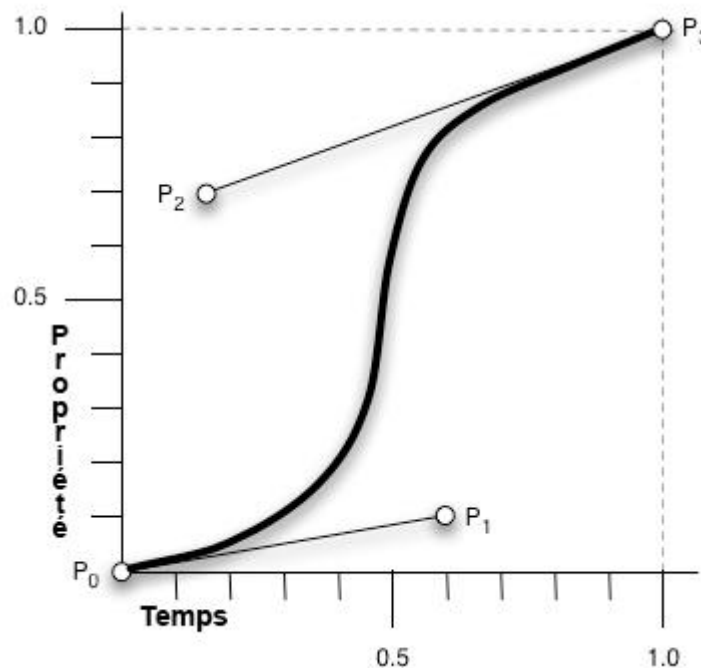
Remarque: Chaque effet par défaut correspond à une fonction cubic-bezier() ou steps(). J'indique les valeurs de chaque fonction à titre de référence, nous nous attarderons plus en détail à l'utilisation plus poussée des fonctions cubic-bezier() et steps() dans les prochains chapitres.

La fonction CSS cubic-bezier()

La syntaxe

Une courbe de cubic-bezier sera définie par quatre points (p_0 , p_1 , p_2 , p_3) disposés sur un graphique où les axes X et Y auront pour valeur une échelle entre 0 et 1.

Les points p_0 et p_3 auront toujours pour valeur de positionnement respective (0,0) et (1,1). En soit, le p_0 représente le début de la transition et le p_3 représente la fin. Ces deux points sont fixes et ne peuvent pas être modifiés.



Les p_1 et p_2 en revanche pourront être positionnés selon nos besoins sur le graphique à l'aide de la fonction cubic-bezier.

Ainsi, la fonction cubic-bezier nous sert à définir la position en X et en Y des points 1 et 2. La fonction prendra en paramètre 4 nombres flottants représentant respectivement:

transition-timing-function: cubic-bezier(<X p1>, <Y p1>, <X p2>, <Y p2>);

Remarque : Il faut obligatoirement utiliser un nombre flottant. Nous devons donc écrire 1.0 en lieu et place de 1, et ainsi de suite.

Outil permettant de réaliser des courbes de Bézier en ligne:
<http://cubic-bezier.com/>

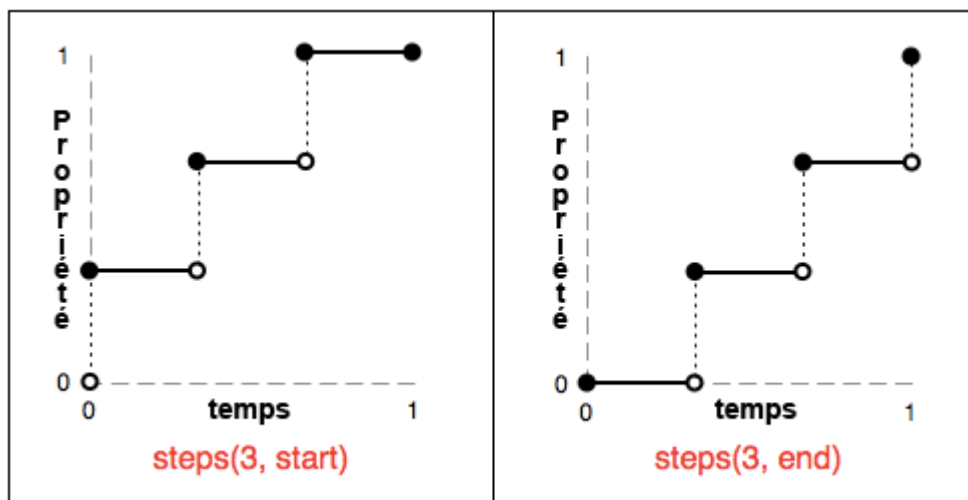
La fonction steps()

La fonction CSS steps() vous permet de définir une transition en "escalier". Cela veut dire que votre transition ne sera pas fluide comme si elle utilisait cubic-bezier, mais passera par des étapes (step) bien tranchées.

La syntaxe de cette fonction est plutôt simple. Elle prendra deux paramètres :

- Premier paramètre: Un nombre entier représentant le nombre de "marche" de la transition.
- Deuxième paramètre: Un mot clé qui définira si la propriété subissant la transition sera modifiée au début ou à la fin du délai de chaque marche. Le mot clé sera start(au début) ou end (à la fin).

Ainsi, le graphique représentant les fonctions steps(3,start) et steps(3,end) serait le suivant:



La fonction steps() pourrait être utilisé pour imiter l'action d'une machine à écrire, ou alors pour animer une image.

A propos de CSS3 Grid Layout

Le concept général de Grid Layout (ou "positionnement en grille") est de diviser virtuellement l'espace en zones majeures dans une page ou une application.

Concrètement et schématiquement, il s'agira de découper en lignes et en colonnes notre page, comme nous le ferions pour un tableau de mise en page.

Malheureusement les navigateurs ne tolèrent que très mal ce nouveau concept. Opéra, Safari et Firefox sur Desktop ne le tolèrent pas et sur mobile seul IE le tolère.

Réaliser un fond dégradé sans image

La propriété utilisée est **background-image**

Les valeurs:

linear-gradient()

radial-gradient()

On définit le type de dégradé voulu : linéaire (linear-gradient) ou radial (radial-gradient) ;

On indique les couleurs de départ et d'arrivée et, éventuellement, leurs proportions de mélange.

Ne rien indiquer comme proportions revient à déclarer (white 0%, black 100%), ce qui donne le mélange maximum entre les deux couleurs. Le fondu atteint ses limites on déclarant 50% aux deux couleurs (white 50%, black 50%) ou toute autre quantité où le % de la couleur initiale est supérieur à celui de la couleur finale.

Toujours par défaut, le dégradé se fait vers le bas, ce qui équivaut à déclarer (to bottom, white, black).

Site de génération de dégradé:

Exemples:

```
.degrade1
{
background-image:linear-gradient(to right, white, black);
}

.degrade2
{
background-image:-moz-linear-gradient(30deg, white, black);
background-image:-webkit-linear-gradient(30deg, white, black);
background-image:-o-linear-gradient(30deg, white, black);
background-image:linear-gradient(60deg, white, black);
}
```

Remarque: IE ne l'interprète qu'à partir de IE 10

Mise en place des media queries

Les media queries font partie des nouveautés de CSS3. Il ne s'agit pas de nouvelles propriétés mais de règles que l'on peut appliquer dans certaines conditions.

Concrètement, vous allez pouvoir dire « Si la résolution de l'écran du visiteur est inférieure à tant, alors on applique les propriétés CSS suivantes ». Cela vous permet de changer l'apparence du site dans certaines conditions : vous pourrez augmenter la taille du texte, changer la couleur de fond, positionner différemment votre menu dans certaines résolutions, etc.

Contrairement à ce qu'on pourrait penser, les media queries ne concernent pas que les résolutions d'écran. Vous pouvez changer l'apparence de votre site en fonction d'autres critères comme le type d'écran (smartphone, télévision, projecteur...), le nombre de couleurs, l'orientation de l'écran (portrait ou paysage), etc. Les possibilités sont très nombreuses !

Les media queries fonctionnent sur tous les navigateurs récents, y compris Internet Explorer à partir de la version 9

Appliquer une media query

Les **media queries** sont donc des règles qui indiquent quand on doit appliquer des propriétés CSS. Il y a deux façons de les utiliser :

- En chargeant une feuille de style .css différente en fonction de la règle (ex : « Si la résolution est inférieure à 1280px de large, charge le fichier petite_resolution.css ») ;
- En écrivant la règle directement dans le fichier .css habituel (ex : « Si la résolution est inférieure à 1280px de large, charge les propriétés CSS ci-dessous »).

Chargement d'une feuille de style différente

Code : HTML

```
<link rel="stylesheet" href="style.css" />
```

On peut lui ajouter un attribut media, dans lequel on va écrire la règle qui doit s'appliquer pour que le fichier soit chargé. On dit qu'on fait une « requête de media » (media query en anglais).

Exemple :

Code : HTML

Si nous utilisons des feuilles de style différentes

```
<link rel="stylesheet" media="screen and (max-width: 1280px)"
href="petite_resolution.css" />
```

Si nous utilisons les media queries au sein de la même feuille de style

```
@media (orientation: portrait), (min-width: 700px) { body { width: 680px}}
@media (orientation: portrait), (max-width: 600px) { body { width : 100%}}
```

Il existe de nombreuses règles permettant de construire des media queries, ici les principales :

color : gestion de la couleur (en bits/pixel).

height : hauteur de la zone d'affichage (fenêtre).

width : largeur de la zone d'affichage (fenêtre).

device-height : hauteur du périphérique.

device-width : largeur du périphérique.

orientation : orientation du périphérique (portrait ou paysage).

media : type d'écran de sortie.

Quelques-unes des valeurs possibles :

screen : écran « classique » ;

handheld : périphérique mobile ;

print : impression ;

tv : télévision ;

projection : projecteur ;

all : tous les types d'écran.

On peut rajouter le préfixe min- ou max- devant la plupart de ces règles. Ainsi, min-width signifie « Largeur minimale », max-height « Hauteur maximale », etc.

Les règles peuvent être combinées à l'aide des mots suivants :

only : « uniquement » ;

and : « et » ;

not : « non ».

le : « **or** » n'existe pas mais peut être écrit en utilisant la virgule, comme suit:

@media (orientation: portrait), (min-width: 600px)

Le meta viewport pour les mobiles

Le **Viewport** désigne schématiquement la surface de la fenêtre du navigateur. Cependant, la notion de **viewport** sur un appareil mobile est différente de celle sur un écran de bureau : sur mobile, le navigateur ne dispose pas de "fenêtre" réelle, ni de barres de défilement car tout est prévu pour naviguer de manière tactile.

Flexbox ! Première version ! En annexe la seconde version

Le modèle de boîte flexible

Sous l'appellation ***Flexible box model module***, CSS 3 signe son projet le plus avancé et prometteur puisque déjà compris par un certain nombre de navigateurs récents.

Le modèle de boîte flexible apparaît comme une extension du modèle de boîte classique défini en CSS 2.1, bénéficiant bien entendu de nouveaux potentiels, parmi lesquels la possibilité d'alterner entre une distribution horizontale ou verticale des éléments, disposer de largeurs fluides dans les deux sens et, surtout, de pouvoir définir l'ordre exact d'affichage des boîtes à l'écran.

display : box

Le modèle de boîte étendu est initié par la valeur box associée à une propriété que nous connaissons bien à présent : display. Il vous suffit d'affecter la déclaration display: box à un bloc pour lui faire adopter le modèle de boîte flexible et que ses éléments se disposent automatiquement les uns à côté des autres.

Exemple:

Partie HTML

```
<div id="main">
<p>une boîte</p>
<p>encore une boîte</p>
<p>et encore une troisième boîte</p>
<p>et hop!</p>
</div>
```

Partie CSS

```
#main
{
display: box;
}
```



Dans la pratique, ce module n'étant pas finalisé, il conviendra d'utiliser les préfixes -moz- pour Mozilla Firefox et -webkit- pour Chrome et Safari, ce qui nous donne :

Partie CSS

```
#main
{
display: -moz-box;
display: -webkit-box;
display: box;
}
```

Empilement vertical ou horizontal

La propriété `box-orient` fait basculer les boîtes d'une distribution horizontale vers un empilement vertical

Exemple

```
#main
{
display: -moz-box;
display: -webkit-box;
display: box;
-moz-box-orient: vertical;
-webkit-box-orient: vertical;
box-orient: vertical;
}
```

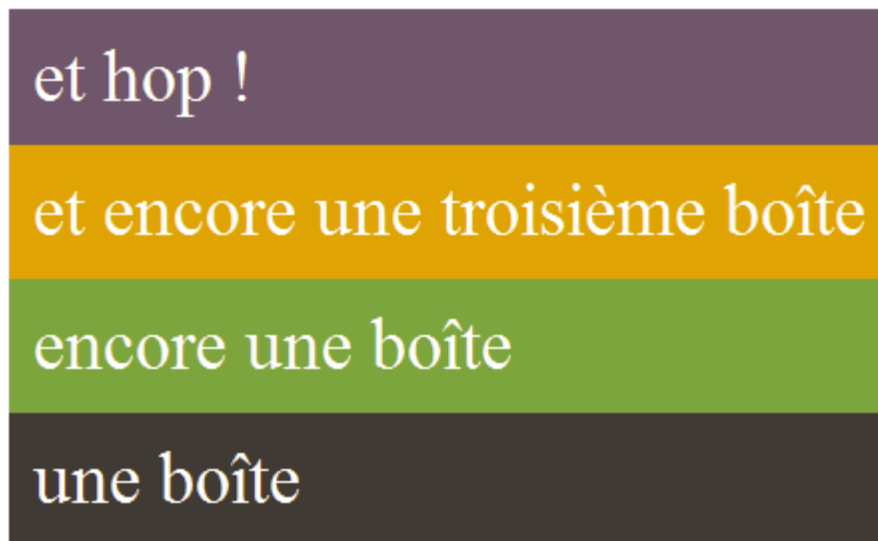


Ordre d'empilement

Comptant parmi les fonctionnalités les plus attractives de ce module, la propriété `box-direction` permet de trier les boîtes selon l'ordre d'apparition dans le flux ou selon l'ordre inverse

(figure 5-31) :

```
#main {
display: -moz-box;
display: -webkit-box;
display: box;
-moz-box-orient: vertical;
-webkit-box-orient: vertical;
box-orient: vertical;
-moz-box-direction: reverse;
-webkit-box-direction: reverse;
box-direction: reverse;
}
```



Il est même possible de définir explicitement l'ordre de distribution des boîtes grâce à la propriété `box-ordinal-group`. La distribution se fait du plus petit (le groupe 1) au plus grand (le groupe 2, puis 3, etc.). La valeur par défaut de cette propriété est 1.

```
#main > p
{
-moz-box-ordinal-group: 2;
-webkit-box-ordinal-group: 2;
box-ordinal-group: 2;
}
```

Positionnement avancé

```
#main > p:last-child {
-moz-box-ordinal-group: 1;
-webkit-box-ordinal-group: 1;
box-ordinal-group: 1;}
```

Cet exemple définit un groupe ordinal secondaire (valeur 2) pour chaque boîte de paragraphe, à l'exception de la dernière qui, par conséquent, s'affichera... en premier

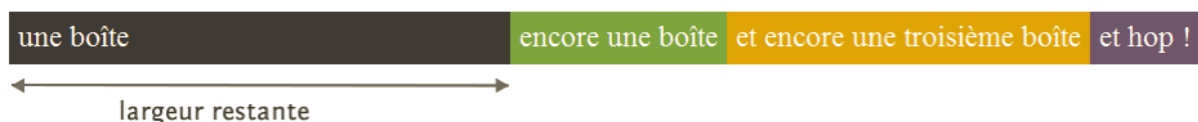


Flexibilité : remplir l'espace

Comme son nom l'indique, le modèle de boîte flexible apporte en toute simplicité une dimension de fluidité aux éléments.

Une boîte devient flexible à partir du moment où elle se voit attribuer la propriété `box-flex` avec une valeur supérieure ou égale à 1. Elle s'étend alors sur toute la surface disponible restante au sein de son parent

```
#main
{
  ...
  width: 100%;
}
#main > p:first-child
{
  -moz-box-flex: 1;
  -webkit-box-flex: 1;
  box-flex: 1;
}
```



Lorsque plusieurs éléments bénéficient de la même valeur de `box-flex`, ils se répartissent équitablement l'espace entre eux. La valeur de `box-flex` correspond à un rapport : une valeur de 2 sera deux fois plus large qu'une valeur de 1 :

```
#main > p
{
-moz-box-flex: 1;
-webkit-box-flex: 1;
box-flex: 1;
}
```

```
#main > p:first-child
{
-moz-box-flex: 2;
-webkit-box-flex: 2;
box-flex: 2;
}
```

Tableau des compatibilités

Navigateurs	Versions	Détails
	Internet Explorer 10+ IE mobile 10+	- Ancienne spécification pour IE10 (2011). Avec préfixe -ms- - Spécification finale sans préfixe pour IE11
	Firefox 2+	- Ancienne spécification (2009). Avec préfixe-moz- - Spécification finale sans préfixe depuis Firefox 22
	Chrome 4+ Chrome Mobile (Android 4+)	- Ancienne spécification (2009) depuis Chrome 4. Avec préfixe -webkit- - Spécification finale depuis Chrome 21. Avec préfixe -webkit- - Spécification finale sans préfixe depuis Chrome 29. - Ancienne spécification pour Chrome sur Android. Avec préfixe -webkit-
	Opera 12.1+ Opera Mobile 12.1+	- Spécification finale sans préfixe entre les versions 12 et 15 - Spécification finale depuis Opera 16. Avec préfixe -webkit-
	Safari 3.1+ Safari Mobile (iOS 3.2+)	- Ancienne spécification (2009). Avec préfixe-webkit- - Spécification finale depuis Safari 7. Avec préfixe -webkit-
	Android Browser 2.1+	- Ancienne spécification (2009). Avec préfixe-webkit-

Mémento des propriétés CSS3

Propriétés du texte

Propriété	Valeurs (exemples)	Description
font-family	<i>police1, police2, police3</i> , serif, sans-serif, monospace	Nom de police
@font-face	<i>Nom et source de la police</i>	Police personnalisée
font-size	1.3em, 16px, 120%...	Taille du texte
font-weight	bold, normal	Gras
font-style	italic, oblique, normal	Italique
text-decoration	underline, overline, line-through, blink, none	Soulignement, ligne au-dessus, barré ou clignotant
font-variant	small-caps, normal	Petites capitales
text-transform	capitalize, lowercase, uppercase	Capitales
font	-	Super propriété de police. Combine : font-weight, font-style, font-size, font-variant, font-family.
text-align	left, center, right, justify	Alignement horizontal
vertical-align	baseline, middle, sub, super, top, bottom	Alignement vertical (cellules de tableau ou éléments inline-block uniquement)
line-height	18px, 120%, normal...	Hauteur de ligne
text-indent	25px	Alinéa
white-space	pre, nowrap, normal	Césure
word-wrap	break-word, normal	Césure forcée

Propriété	Valeurs (exemples)	Description
text-shadow	5px 5px 2px blue (<i>horizontale, verticale, fondu, couleur</i>)	Ombre de texte

Propriétés de couleur et de fond

Propriété	Valeurs (exemples)	Description
color	<i>nom</i> , rgb(rouge,vert,bleu), rgba(rouge,vert,bleu,transparence), #CF1A20...	Couleur du texte
background-color	<i>Identique à color</i>	Couleur de fond
background-image	url('image.png')	Image de fond
background-attachment	fixed, scroll	Fond fixe
background-repeat	repeat-x, repeat-y, no-repeat, repeat	Répétition du fond
background-position	(x y), top, center, bottom, left, right	Position du fond
background	-	Super propriété du fond. Combine :background-image, background-repeat,background-attachment, background-position
background-size	"px" ou "%" "cover" : forcera à couvrir toute la surface sans déformer l'image. Quitte à la rogner. "contain" : forcera l'image à ne pas dépasser de l'élément sans la déformer.	Taille de l'image dans l'arrière plan
opacity	0.5	Transparence

Propriété	Valeurs (exemples)	Description
Remarque:		
- La css3 permet des background multiples. - Attention en cas de superposition, le résultat est similaire à des calques d'un logiciel graphique tel que Photoshop : l'image la plus proche de la propriété (la première énumérée) sera l'image de premier plan. Si une couleur de fond est déclarée, elle sera toujours reléguée au dernier plan.		
Exemple:		
<pre>#example1 { width: 500px; height: 250px; background-image: url(coucou.png), url(coucou2.png); background-position: center bottom, left top; background-repeat: no-repeat; }</pre>		

Propriétés des boîtes

Propriété	Valeurs (exemples)	Description
width	150px, 80%...	Largeur
height	150px, 80%...	Hauteur
min-width	150px, 80%...	Largeur minimale
max-width	150px, 80%...	Largeur maximale
min-height	150px, 80%...	Hauteur minimale
max-height	150px, 80%...	Hauteur maximale
margin-top	23px	Marge en haut
margin-left	23px	Marge à gauche
margin-right	23px	Marge à droite

Propriété	Valeurs (exemples)	Description
margin-bottom	23px	Marge en bas
margin	23px 5px 23px 5px (haut, droite, bas, gauche)	Super-propriété de marge. Combine : margin-top, margin-right, margin-bottom, margin-left.
padding-left	23px	Marge intérieure à gauche
padding-right	23px	Marge intérieure à droite
padding-bottom	23px	Marge intérieure en bas
padding-top	23px	Marge intérieure en haut
padding	23px 5px 23px 5px (haut, droite, bas, gauche)	Super-propriété de marge intérieure. Combine : padding-top, padding-right, padding-bottom, padding-left.
border-width	3px	Épaisseur de bordure
border-color	nom, rgb(rouge,vert,bleu), rgba(rouge,vert,bleu,transparence), #CF1A20...	Couleur de bordure
border-style	solid, dotted, dashed, double, groove, ridge, inset, outset	Type de bordure

Propriété	Valeurs (exemples)	Description
border	3px solid black	Super-propriété de bordure. Combine border-width, border-color, border-style. Existe aussi en version border-top, border-right, border-bottom, border-left.
border-radius	5px	Bordure arrondie
box-shadow	6px 6px 0px black (<i>horizontale, verticale, fondu, couleur</i>)	Ombre de boîte

Propriétés de positionnement et d'affichage

Propriété	Valeurs (exemples)	Description
display	block, inline, inline-block, table, table-cell, none...	Type d'élément (block, inline, inline-block, none...)
visibility	visible, hidden	Visibilité
clip	rect (0px, 60px, 30px, 0px) <i>rect (haut, droite, bas, gauche)</i>	Affichage d'une partie de l'élément
overflow	auto, scroll, visible, hidden	Comportement en cas de dépassement
float	left, right, none	Flottant
clear	left, right, both, none	Arrêt d'un flottant
position	relative, absolute, static	Positionnement
top	20px	Position par rapport au haut
bottom	20px	Position par rapport au bas
left	20px	Position par rapport à la gauche
right	20px	Position par rapport à la droite
z-index	10	Ordre d'affichage en cas de superposition. La plus grande valeur est affichée par-dessus les autres.

Propriétés des listes

Propriété	Valeurs (exemples)	Description
list-style-type	disc, circle, square, decimal, lower-roman, upper-roman, lower-alpha, upper-alpha, none	Type de liste
list-style-position	inside, outside	Position en retrait
list-style-image	url('puce.png')	Puce personnalisée
list-style	-	Super-propriété de liste. Combine list-style-type, list-style-position, list-style-image.

Propriétés des tableaux

Propriété	Valeurs (exemples)	Description
border-collapse	collapse, separate	Fusion des bordures
empty-cells	hide, show	Affichage des cellules vides
caption-side	bottom, top	Position du titre du tableau

Autres propriétés

Propriété	Valeurs (exemple)	Description
cursor	crosshair, default, help, move, pointer, progress, text, wait, e-resize, ne-resize, auto...	Curseur de souris